

**Simulation d'un système de drone autonome pour la recherche
et le sauvetage : Contrôle aérien, vision artificielle et
reconstruction 3D en environnement virtuel**



QIAN Boda

HUANG Bowen

NDANKOU BOBO THIERRY

M1-EEEE

Ali AMOURI

Enseignant-Professeur

26/01/2026 – 06/05/2026

Table des matières

1. Introduction	5
1.1 Présentation du sujet de recherche.....	5
1.2 Contexte de l'application du drone dans la recherche et le sauvetage (R&S).....	5
1.3 Objectifs du projet et contenu principal du travail	6
2. Etude theorique reseau 5G.....	7
2.1 introduction	7
2.1.1 Notion d'environnement multi-agent et coordination	7
2.2.2 Exigences de communication pour la coordination et pourquoi la 5G.....	7
2.2 Le réseau 5G.....	8
2.2.1 Couche d'accès radio (RAN – Radio Access Network)	8
2.2.2 Qualité de service (QoS) et gestion dynamique	9
2.3 Modéliser les mécanismes de coordination	9
2.3.1 Objectif de la coordination dans les systèmes de drones.....	9
2.4 Analyser des performances attendues : latence, fiabilité, débit, passage à l'échelle .	10
2.4.1 Introduction	10
2.4.2. Latence	11
2.4.3 Fiabilité.....	11
2.4.4 Débit	11
2.4.5. Passage à l'échelle (Scalabilité)	12
2.5 Limites de la 5G : congestion, dépendance à la couverture, sécurité et synchronisation.....	12
2.5.1. Introduction	12
2.5.2. Congestion/saturation du réseau.....	12
2.5.3 Dépendance à la couverture et mobilité aérienne	13
2.5.4 Sécurité et intégrité des communications	13

3. État de l’art & Fondements théoriques	14
3.1 Aperçu des systèmes de drones autonomes pour la R&S:	14
3.2 Outils & technologies utilisées:	14
3.3 Bases théoriques du drone quadrotor.....	15
3.3.1 Modélisation de drone	15
3.3.2 Système de coordonnées de repère et définition des symboles	15
3.3.3 Matrice de rotation	16
3.3.4 Modèle cinématique	17
3.3.5 Modèle cinétique	17
3.3.6 Modèle de performance de commande.....	18
3.3.7 Modèle de groupe moteur.....	18
3.3.8 Modèle globale de simulation du drone sur simulink.....	20
3.4 Principes de base du contrôle de vol	20
3.4.1 Le schéma du contrôleur et du modèle linéarisé	21
3.4.2 Construction du modèle de contrôle basée sur Simulink.....	22
3.4.3 Simulation de contrôle de drone basée sur Simulink	25
4. Mise en place de la plateforme de simulation et communication entre modules	28
4.1 Présentation brief des outils de simulation	28
4.1.1 PX4 Autopilot.....	28
4.1.2 AirSim	28
4.1.3 ROS2	29
4.1.4 QGroundControl (QGC).....	30
4.2 Détails de la communication entre modules.....	30
4.3 Mise en place de l’environnement de simulation et tests de coordination	33
4.3.1 Processus d’installation et de configuration de l’environnement.....	33
4.3.2 Tests de coordination et de communication entre les modules.....	34
4.3.3 Validation de l’efficacité de la simulation de vol de base du drone	34

5. Développement et mise en œuvre des fonctionnalités du système.....	37
5.1 Intégration des capteurs simulés.....	37
5.1.1 Configuration de la caméra, du LiDAR et de la caméra de profondeur dans AirSim AirSim.....	37
5.1.2 Visualisation des données des capteurs basée sur Rviz	38
5.2 Détection de la recherche et du sauvetage humain par vision artificielle.....	40
5.2.1 Présentation du principe des algorithmes YOLO et OpenCV	40
5.2.2 Pipeline de détection dans un environnement simulé.....	40
5.2.3 Mise en œuvre de l'effet de détection de piétons dans l'environnement virtuel	41
5.3 Modélisation de la reconstruction 3D de l'environnement.....	42
5.3.1 Principe de la cartographie 3D avec OctoMap.....	42
5.3.2 Processus de conversion d'image de profondeur en nuage de points et reconstruction de carte.....	42
5.3.3 Mise en œuvre de l'effet de reconstruction 3D de la scène simulée.....	43
6. Résultats expérimentaux et validation des performances du système	45
6.1 Résultats de simulation du contrôle de vol du drone.....	45
6.2 Résultats des performances de détection d'objets humains.....	45
6.3 Effet de reconstruction 3D et analyse de précision	46
7. Conclusion générale	48
8. Répartition des tâches de l'équipe et page de contribution individuelle	50
8.1 Répartition des tâches au sein de l'équipe.....	50
8.2 Contribution individuelle.....	51
QIAN Boda.....	51
HUANG Bowen	52
Thierry NDANKOU BOBO.....	53
RÉFÉRENCES	54

1. Introduction

1.1 Présentation du sujet de recherche

Le sujet de cette recherche porte sur la simulation d'un système de drone autonome dédié à la recherche et au sauvetage (R&S) dans un environnement virtuel, intégrant les travaux collaboratifs de notre groupe sur la simulation et la modélisation réseau. Dans le contexte actuel où les missions de R&S font face à de nombreux défis — environnements hostiles, risques pour la sécurité des sauveteurs et faible efficacité de la recherche traditionnelle — le drone autonome est devenu un outil indispensable dans le domaine de la R&S[1]. Ce projet se concentre sur l'intégration de technologies clés : contrôle aérien précis, détection humaine (YOLO, OpenCV) et reconstruction 3D (OctoMap). Nous étudions également la modélisation du drone et les bases du contrôle de vol pour soutenir la simulation et le vol réel. Nous utilisons MATLAB/Simulink, FlightGear et Webots pour la simulation, réalisons une modélisation 5G via des agents robotiques, et nous appuyons sur PX4, AirSim, ROS2 et QGC pour développer une plateforme fiable et valider les performances du système de drone autonome pour la R&S. Le code source du projet ainsi que les vidéos de démonstration sont consultables sur notre dépôt GitHub : <https://github.com/BodaQian/ProjectTER>.

1.2 Contexte de l'application du drone dans la recherche et le sauvetage (R&S)

Avec l'accélération de l'urbanisation et l'augmentation fréquente des catastrophes naturelles (inondations, sécheresses, séismes) et des accidents humains, les demandes pour les missions de R&S sont en constante augmentation. La méthode de R&S traditionnelle, qui dépend principalement de l'intervention manuelle des sauveteurs, présente des limites évidentes : longue durée de recherche, risque élevé pour la sécurité des sauveteurs, et incapacité à accéder à des zones éloignées ou dangereuses (forêts denses, zones montagneuses, décombres). Par conséquent, ces dernières années, le drone (UAV), grâce à ses nombreux avantages, est devenu une solution économique et efficace pour faire face aux situations d'urgence[2]. Premièrement, le drone peut être déployé rapidement dans les zones sinistrées pour fournir un réseau mobile à la demande ; deuxièmement, il peut atteindre rapidement des lieux inaccessibles, tels que les montagnes, les déserts ou les zones sinistrées, et couvrir de grandes zones de recherche avec une distribution éparse de victimes.

Sur la base de ces avantages, le drone autonome a été largement appliqué dans le domaine de la R&S. Cependant, la validation et l'optimisation du système de drone autonome dans des scénarios réels sont non seulement coûteuses et risquées, mais également difficiles à reproduire pour diverses scènes complexes de R&S. Pour cela, la simulation virtuelle devient une méthode

efficace pour développer et valider le système de drone autonome, qui peut non seulement réduire les coûts de développement et éviter les risques d'expérimentation, mais également tester complètement les performances du système dans diverses scènes simulées, posant une base pour les applications pratiques ultérieures.

1.3 Objectifs du projet et contenu principal du travail

Pour atteindre ces objectifs, le travail principal de notre groupe se divise en plusieurs étapes : d'abord, nous effectuons la Modélisation Dynamique et Cinématique du drone, la Modélisation du Système de Contrôle, et mettons en œuvre la Simulation de Vol Fondamentale avec MATLAB/Simulink et FlightGear ; ensuite, nous installons et configurons les outils de simulation (PX4, AirSim, ROS2, QGroundControl, Webots) et intégrons les modules ; par la suite, nous développons les fonctionnalités applicatives, incluant l'intégration des capteurs simulés (caméra, LiDAR), la détection de personnes avec YOLO et OpenCV, la reconstruction 3D avec OctoMap et l'étude de la communication 4G/5G ; puis, nous conduisons des tests expérimentaux pour valider les performances du système ; enfin, nous synthétisons les travaux, les résultats et les analyses, et complétons les pages de contribution individuelle et de conclusion.

2. Etude theorique reseau 5G

2.1 introduction

La coordination d'agents autonomes (logiciels, robots, véhicules ou capteurs) est devenue un enjeu majeur de notre société, nécessitant des communications rapides, fiables et synchronisées pour atteindre des objectifs collectifs complexes. Les réseaux 4G, bien que performants, restent limités pour ces applications exigeantes en temps réel. La 5G constitue une avancée décisive en introduisant une architecture flexible basée sur la virtualisation (NFV), le découpage des ressources (network slicing) et le calcul en périphérie (MEC).

Cette évolution favorise l'émergence de systèmes multi-agents capables de s'adapter et de coopérer en temps réel. Toutefois il y a des défis tels que la congestion, la sécurité, la dépendance à la couverture et la synchronisation.

2.1.1 Notion d'environnement multi-agent et coordination

Un agent est une entité autonome capable de percevoir son environnement, de traiter l'information et d'agir pour atteindre un objectif. Dans un système multi-agents (SMA), plusieurs agents interagissent de manière coordonnée afin de résoudre collectivement des problèmes complexes. Ces agents peuvent être physiques (robots, drones, véhicules) ou même logiciels (services de distribution)[3]. Les SMA permettent de décentraliser le contrôle, d'améliorer la résilience et d'exploiter efficacement les ressources distribuées. Toutefois, leur performance dépend fortement de la qualité des communications entre agents.

La coordination de son côté correspond aux mécanismes permettant à plusieurs agents d'aligner leurs actions pour atteindre un objectif commun. Elle repose sur trois éléments essentiels : le partage d'information entre agents, la prise de décision distribuée et la gestion des conflits ou redondances. Selon l'organisation du système, elle peut être centralisée (avec un coordinateur unique ; le père), décentralisée (décisions locales basées sur des informations partielles) ou hybride. Grâce à son architecture hiérarchisée (edge-core-cloud) et à ses capacités de virtualisation, la 5G constitue un support particulièrement adapté à la mise en œuvre de ces modèles de coordination, en offrant un bon compromis entre réactivité et cohérence globale.

2.2.2 Exigences de communication pour la coordination et pourquoi la 5G

La performance d'une coordination multi-agents repose directement sur les caractéristiques du réseau supportant les échanges entre agents. Dans une architecture 5G, plusieurs exigences clés

apparaissent comme la fiabilité élevée, synchronisation et latence ultra faible pour ne citer que les plus pertinents. Au-delà des performances brutes, la 5G introduit une possibilité inédite : adapter dynamiquement les ressources de communication au besoin contextuel de chaque groupe d'agents grâce au *network slicing*. Ce découpage virtuel permet, par exemple, d'allouer un sous-réseau hautement fiable et à très faible latence à un essaim de drones tout en maintenant des canaux massifs, mais moins prioritaires, pour des capteurs IoT environnants[4].

2.2 Le réseau 5G

La 5G constitue une évolution technologique majeure en répondant à trois types d'usages :

- Le haut débit mobile (eMBB : enhance mobile broadband),
- les communications ultra fiables à faible latence (URLLC : Ultra Reliable and Low Latency Communications)
- La connectivité massive d'objets (mMTC : massive Machine Type Communication).

Avec ces 3 grands atouts, elle permet ainsi de combiner performance, réactivité et passage à l'échelle.

L'architecture de la 5G repose sur la séparation entre le plan de contrôle et le plan utilisateur, favorisant la virtualisation et l'allocation dynamique des ressources réseau. Dans le contexte des systèmes de drones, cette flexibilité permet d'adapter le réseau aux besoins spécifiques : par exemple ; utiliser un slice URLLC pour le contrôle en temps réel et un slice eMBB pour la transmission vidéo, optimisant ainsi la coordination et les performances globales.

2.2.1 Couche d'accès radio (RAN – Radio Access Network)

La couche d'accès radio de la 5G introduit des innovations majeures :

- Massive MIMO (Multiple Input Multiple Output) : utilisation d'un grand nombre d'antennes pour améliorer le débit, la portée et la robustesse du lien radio.
- Beamforming adaptatif : ciblage directionnel du signal, particulièrement utile pour des drones en mouvement rapide, réduisant les pertes de liaison.
- Découpage des bandes de fréquences : nouvelles gammes de fréquences (Sub-6 GHz, mmWave, bandes FR1 et FR2) permettant d'ajuster le débit et la portée selon les conditions.
- Split du RAN (CU/DU/RU) : séparation fonctionnelle en *Central Unit* (CU), *Distributed Unit* (DU) et *Radio Unit* (RU), rendant la gestion du réseau radio plus flexible et adaptée à la mobilité aérienne.

Ainsi, pour un essaim de drones, la 5G peut maintenir des liens stables malgré la mobilité et la densité d'appareils, ce qu'aucune génération précédente ne permettait à ce niveau.

2.2.2 Qualité de service (QoS) et gestion dynamique

Le modèle de QoS 5G dépasse celui des générations précédentes grâce à une granularité accrue :

- Mécanismes d'allocation adaptatifs (Dynamic QoS Flow)
- Priorisation selon le niveau de criticité (ex. commande de drone > vidéo de surveillance) ;
- Gestion proactive de la congestion par intelligence artificielle.

Les fonctions de gestion de ressources radio (RRM) et de stratégie (Policy Control Function – PCF) supervisent l'attribution de bande passante et la commutation dynamique entre slices.

Le cœur de réseau 5G repose sur une logique de fonctions virtualisées (Network Functions – NFs) interconnectées par des API RESTful. Les principales composantes sont [5]:

- AMF (Access and Mobility Management Function) : gère la mobilité et l'attachement réseau ;
- SMF (Session Management Function) : supervise les sessions de données
- UPF (User Plane Function) : traite le trafic utilisateur
- PCF, NEF, NRF, AUSF : contrôlent respectivement les politiques, les expositions de service, le registre fonctionnel et la sécurité.

Cette architecture orientée services (*Service-Based Architecture*, SBA) assure la souplesse, la programmabilité et l'orchestration multi-niveau indispensables aux applications aériennes et industrielles.

2.3 Modéliser les mécanismes de coordination

2.3.1 Objectif de la coordination dans les systèmes de drones

La coordination entre drones permet de réaliser des missions complexes où la coopération améliore la performance globale : exploration simultanée, livraison collective, surveillance, recherche et sauvetage.

Chaque drone agit en agent autonome, mais doit échanger des informations essentielles (position, vitesse, charge utile, état de capteurs, etc.) afin de maintenir la cohérence du groupe.

Dans un environnement 5G, la coordination devient un problème interdisciplinaire de la communication sans fil, du traitement distribué (intelligence artificiel) et de la prise de décision

collective (algorithmes de consensus, apprentissage partagé, planification coordonnée).

On peut avoir plusieurs type de coordination c'est-à-dire la coordination centralisé, décentralisé ou hybride.

- Dans la coordination centralisé on a un contrôleur unique généralement hébergé sur une station de base 5G (gNB) ou un nœud edge. Ce contrôleur collecte les données de l'ensemble des drones et calcule les décisions à appliquer.
- Dans une architecture décentralisée, les drones communiquent directement entre eux via le réseau 5G (V2X / D2D — *Device-to-Device*), sans passer systématiquement par un serveur central. Chaque agent décide localement en fonction des messages reçus de ses voisins. C'est cette architecture que nous allons utiliser dans notre simulation.

Rôle de la 5G dans cette con

uration :

La 5G supporte les communications sidelink (liaisons directes entre terminaux, spécifiées dans la 3GPP Release 16-17), offrant des débits stables et une portée améliorée pour les drones en formation serrée. Les fonctions de *network slicing* permettent d'allouer un canal dédié aux échanges inter-drones, séparé du trafic de commande.

Cas typique : Formation de drones de livraison coordonnés par apprentissage multi-agent, partageant en temps réel leur état de vol et coordonnant la répartition des charges sans supervision centrale.

- La coordination hybride peut être implémenté en combinant une supervision centrale pour la cohérence stratégique et une coordination locale entre drones pour la réactivité.

2.4 Analyser des performances attendues : latence, fiabilité, débit, passage à l'échelle

2.4.1 Introduction

Les performances de la coordination de drones dépendent étroitement de la qualité des communications offertes par la 5G. Dans un système distribué, chaque paramètre latence, fiabilité, vitesse de transfert ou capacité d'évolution a un impact direct sur la stabilité, la réactivité et la sécurité des missions.

L'objectif de cette section est d'analyser ces différents indicateurs et de les relier aux mécanismes techniques internes à l'architecture 5G (URLLC, slicing, MEC, beamforming, etc.).

2.4.2. Latence

La latence désigne le délai total entre l'émission d'une information et sa réception.

Dans le pilotage et la coordination d'un essaim de drones, elle conditionne :

- La synchronisation des trajectoires,
- La réactivité aux changements d'environnement
- La sécurité des vols groupés.

Application drone :

Une latence inférieure à 10 ms permet un contrôle en boucle fermée entre le centre de commande et des drones en vol.

2.4.3 Fiabilité

La fiabilité traduit la probabilité qu'un paquet de données soit livré correctement et dans le délai prévu. Pour les communications critiques de drones (commande, synchronisation, sécurité), une fiabilité supérieure à 99,999 % est requise.

En 5G :

- Mécanismes de duplication de paquets
- Réaffectation instantanée de bande
- Multi-connectivité (*Dual Connectivity*) avec plusieurs cellules en parallèle
- Gestion proactive des interférences via IA intégrée au RAN.

Dans un essaim, une telle fiabilité évite les déconnexions imprévues qui pourraient désorganiser la formation ou provoquer des collisions.

2.4.4 Débit

Le débit utile conditionne la quantité d'informations échangées par unité de temps. Pour les drones, il concerne principalement :

- La transmission vidéo HD/4K,
- Les flux de télémétrie et de commande,
- Le transfert de paramètres d'intelligence embarquée.

Bénéfices pour les drones :

- Flux vidéo en temps réel vers des centres de supervision ;

- Apprentissage collaboratif (partage de modèles IA entre drones) ;
- Télémétrie massive pour la coordination prédictive.

2.4.5. Passage à l'échelle (Scalabilité)

L'un des apports majeurs de la 5G réside dans sa capacité à supporter un grand nombre de connexions simultanées.

La densité de connexion peut atteindre jusqu'à 1 million d'appareils par km² en mode mMTC — une condition essentielle pour les environnements saturés de drones, notamment dans les zones urbaines ou les missions collaboratives.

Les mécanismes clés qui assurent cette scalabilité :

- Slicing dynamique répartissant les ressources entre différents groupes de drones ;
- Allocation adaptative du spectre selon la charge locale
- Planification radio intelligente (AI-driven RRM) pour éviter la congestion
- Contrôle distribué via edge orchestrators, permettant une coordination modulaire de sous-essaims.

Dans la pratique, cela se traduit par la possibilité de coordonner des centaines de drones sans perte significative de performance, à condition que la planification des slices et la répartition des rôles soient optimisées.

2.5 Limites de la 5G : congestion, dépendance à la couverture, sécurité et synchronisation

2.5.1. Introduction

Malgré ses capacités remarquables, la 5G ne constitue pas une solution parfaite pour la coordination d'essaims de drones.

Les contraintes liées à la topologie du réseau, à la gestion des ressources et à la sécurité peuvent affecter la performance globale : pertes de paquets, saturation du canal, latence fluctuante, ou encore désynchronisation des agents.

Ici, on examine les principales limites observées.

2.5.2. Congestion/saturation du réseau

La congestion survient lorsque la demande en bande passante excède les ressources radio disponibles dans une cellule.

Ce problème peut être accentué par :

- Une forte densité de drones dans une même zone,
- La coexistence avec d'autres utilisateurs 5G (véhicules, IoT),
- La priorisation dynamique des slices.

Effets :

- Augmentation de la latence
- Pertes de paquets
- Dégradation de la qualité vidéo ou des messages de commande.

2.5.3 Dépendance à la couverture et mobilité aérienne

La continuité du lien 5G n'est pas encore pleinement garantie dans l'espace aérien car les stations ne sont pas encore très optimisées pour terminaux aérien ce qui entraîne une diminution du signal à des altitudes supérieures à 300 m.

2.5.4 Sécurité et intégrité des communications

La sécurité des échanges est particulièrement critique dans les systèmes de drones connectés : une compromission du lien de commande ou de la synchronisation pourrait entraîner une perte de contrôle collective.

Vulnérabilités majeures :

- Interception des flux (attaque *man-in-the-middle*)
- Usurpation d'identité d'un drone ou d'un contrôleur
- Saturation volontaire du canal radio
- Injection de fausses données de navigation.

3. État de l'art & Fondements théoriques

3.1 Aperçu des systèmes de drones autonomes pour la

R&S:

Avec l'augmentation fréquente des catastrophes naturelles et des accidents humains, les systèmes de drones autonomes pour la R&S ont été largement étudiés et appliqués dans le domaine de l'urgence. Actuellement, les recherches mondiales se concentrent principalement sur l'intégration de la commande de vol autonome, de la détection visuelle, de la reconstruction 3D et de la communication sans fil, visant à améliorer l'efficacité et la fiabilité des missions de R&S. Plusieurs chercheurs ont apporté des contributions importantes à ce domaine : Antonio Albanese et ses collaborateurs, ont proposé la solution SARDO, qui exploite les téléphones portables pour localiser les victimes sans support d'infrastructure, avec une précision élevée et un faible consommation de batterie.[6] Naser Alsaedi et son équipe ont développé un cadre de sauvetage basé sur drone intégrant l'IA, déployé sur des appareils de bord et utilisant ROS2 et MAVLink pour assurer une détection rapide des personnes en détresse.[7] Boddu Sanjana et ses collaborateurs ont présenté un cadre basé sur YOLOv8, combinant le flux optique de Farneback pour améliorer la précision de détection et générer des cartes de chaleur pour prioriser les sauvetages.[8] De telles recherches sont diverses et riches, et notre groupe, en se basant sur ces travaux existants, a mis en place une plateforme de simulation de drone intégrant PX4, AirSim, ROS2 et d'autres outils, réalisé la simulation de fonctionnalités clés telles que la détection humaine et la reconstruction 3D, approfondi la compréhension des théories de modélisation drone et de contrôle de vol, et effectué la simulation de la communication 5G pour compléter le système de R&S par drone.

3.2 Outils & technologies utilisées:

Pile de simulation : PX4, AirSim, ROS2, FlightGear, MATLAB et QGroundControl, Webots. Ces outils sont utilisés conjointement pour construire l'environnement de simulation du drone, réaliser le contrôle de vol, configurer les paramètres du drone et valider les performances du système.

Vision et reconstruction 3D : YOLO, OpenCV et OctoMap. YOLO et OpenCV sont appliqués à la détection humaine par vision artificielle, tandis qu'OctoMap est utilisé pour la reconstruction 3D de l'environnement dans les missions de R&S.

Protocoles de communication : MAVLink, Micro-XRCE-DDS, et comparaison de la communication 4G/5G. Ces protocoles garantissent la communication fluide entre les modules du système, et la comparaison de 4G/5G permet d'analyser les performances de communication

dans les scénarios de R&S.

Outils de visualisation : RViz. Il est utilisé pour visualiser les données des capteurs, l'état du drone et les résultats de la reconstruction 3D en temps réel.

3.3 Bases théoriques du drone quadrotor

3.3.1 Modélisation de drone

La modélisation de drone comprend un total de 3 parties : le modèle de l'unité motrice, le modèle d'efficacité de contrôle et le modèle à corps rigide du multirotor, dont le modèle à corps rigide du multirotor inclut le modèle dynamique et le modèle cinématique du corps rigide. Le modèle cinétique fournit des informations d'accélération pour la modélisation cinématique.

Le processus global est le suivant :

Commande des gaz (PWM) - Modèle de la puissance (Vitesse de l'hélice) - Modèle d'efficacité de contrôle (Portance et couple) - Modèle de dynamique des corps rigides (Vitesse et vitesse angulaire) - Modèle cinématique des corps rigides (Position et attitude)

3.3.2 Système de coordonnées de repère et définition des symboles

Système de coordonnées de cellule : Pour les drones quadricoptères, ils sont généralement divisés en racks en forme de X et en cadres en croix. Le système de coordonnées du fuselage est celui du quadricoptère, qui se déplace avec le drone. Il respecte les règles de la main droite.

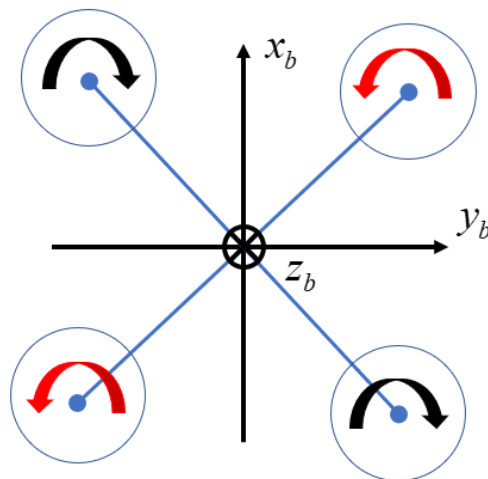


Fig3.3.2 :Schéma du modèle et système de coordonnée de rack UAV quadricoptère.

Système de coordonnées spatiales : Un système de coordonnées fixes dans l'espace, ainsi que la position et la vitesse du drone sont ensuite décrites sous ce système.

Symboles et Définitions de Contrôle de Vol de Drone	
Symbole	Signification
e	Indice, représentant le repère spatial
b	Indice, représentant le repère corps
x_e, y_e, z_e	Repère spatial
x_b, y_b, z_b	Repère corps
ϕ	Angle de roulis, correspondant à la rotation autour de l'axe x_b , équivalent à roll
θ	Angle de tangage, correspondant à la rotation autour de l'axe y_b , équivalent à pitch
ψ	Angle de lacet, correspondant à la rotation autour de l'axe z_b , équivalent à yaw
R_{be}	Matrice de rotation du repère corps au repère spatial
R_{eb}	Matrice de rotation du repère spatial au repère corps
f_b	Traction de l'hélice dans le repère corps, de direction opposée à l'axe z du repère corps
f_e	Traction de l'hélice dans le repère mondial
P	Position du drone dans le repère spatial
V	Vitesse du drone dans le repère spatial $[v_{xe} \ v_{ye} \ v_{ze}]^T$
w_b	Vitesse angulaire du drone dans le repère corps $[w_{xb} \ w_{yb} \ w_{zb}]^T$
Θ_e	Trois angles d'Euler du drone ϕ, θ, ψ
W	Matrice de projection de la vitesse angulaire du repère corps vers le taux de variation des angles d'Euler (également appelée matrice des taux d'angles d'Euler)

3.3.3 Matrice de rotation

Nous définissons la matrice de rotation selon l'ordre de rotation z_b, y_b, x_b du système d'enroulement

$$R(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} R(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix} R(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix}$$

$$R_b^e = R(\psi)R(\theta)R(\phi)$$

$$= \begin{bmatrix} \cos \theta \cos \psi & \cos \psi \sin \theta \sin \phi - \sin \psi \cos \phi & \cos \psi \sin \theta \cos \phi + \sin \psi \sin \phi \\ \cos \theta \sin \psi & \sin \psi \sin \theta \sin \phi + \cos \psi \cos \phi & \sin \psi \sin \theta \cos \phi - \cos \psi \sin \phi \\ -\sin \theta & \sin \phi \cos \theta & \cos \phi \cos \theta \end{bmatrix}.$$

3.3.4 Modèle cinématique

Cinématique positionnelle: P_e est utilisé pour représenter la position de l'UAV dans le système de coordonnées spatiales, V_e sert à représenter la vitesse de l'UAV dans le système de coordonnées spatiales, et la position et la vitesse correspondent à $P_e' = V_e$, et la dynamique fournit des informations d'accélération, donc pour obtenir la position, deux intégrations sont nécessaires.

Cinématique d'attitude: Utilisez w_b pour représenter la vitesse du drone dans le système de coordonnées du fuselage, et utilisez Θ_e pour représenter les trois angles d'Euler ϕ, θ, ψ , nous avons $\Theta_e' = W^* w_b$

$$\Theta = [\phi \quad \theta \quad \psi]^T, W = \begin{bmatrix} 1 & \tan \theta \sin \phi & \tan \theta \cos \phi \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi / \cos \theta & \cos \phi / \cos \theta \end{bmatrix}, w_b = [w_{bx}, w_{by}, w_{bz}]$$

3.3.5 Modèle cinétique

En raison de la rotation de l'hélice, le drone sera affecté par la portance, ce qui modifie la position, la vitesse et l'accélération du drone, et cette portance générera également un moment qui modifie l'attitude du drone (3 angles d'Euler).

Dynamique positionnelle: En ce qui concerne la force à laquelle les drones sont soumis, nous penseront d'abord à la gravité et à la portance des hélices, et en même temps, elles seront également affectées par une certaine traînée. Pour simplifier, seuls la gravité et le rôle de la portance de l'hélice sont pris en compte ici.

Selon $f=ma$, on a $mV_e' = mg - f_e$

Substitution $f_e = R_{be}^* f_b$, on a

$$m\dot{V}_e = mg - R_b^e f_b$$

$$\dot{V}_e = g - R_b^e \frac{f_b}{m}$$

Déplie-le et on l'aura

$$\begin{bmatrix} \dot{v}_{x_e} \\ \dot{v}_{y_e} \\ \dot{v}_{z_e} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} - \begin{bmatrix} \cos \theta \cos \psi & \cos \psi \sin \theta \sin \phi - \sin \psi \cos \phi & \cos \psi \sin \theta \cos \phi + \sin \psi \sin \phi \\ \cos \theta \sin \psi & \sin \psi \sin \theta \sin \phi + \cos \psi \cos \phi & \sin \psi \sin \theta \cos \phi - \cos \psi \sin \phi \\ -\sin \theta & \sin \phi \cos \theta & \cos \phi \cos \theta \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ f_b \end{bmatrix}$$

$$\begin{bmatrix} \dot{v}_{x_e} \\ \dot{v}_{y_e} \\ \dot{v}_{z_e} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} - \frac{f_b}{m} \begin{bmatrix} \cos \psi \sin \theta \cos \phi + \sin \psi \sin \phi \\ \sin \psi \sin \theta \cos \phi - \cos \psi \sin \phi \\ \cos \phi \cos \theta \end{bmatrix}$$

D'après l'équation ci-dessus, on peut voir que pour trouver l'accélération du drone dans le système de coordonnées spatiales, il faut l'accélération gravitationnelle g , m , f_b et trois angles d'Euler (ϕ , θ , ψ).

Dynamique d'attitude: La force de traction générée par l'hélice du drone génère un moment sur les trois axes du système de coordonnées du corps, et il y aura également un moment gyroscopique G a pour l'objet en rotation. Par conséquent, l'équation de la dynamique des attitudes est la suivante : $G_a + \tau = J\dot{w}_b + w_b \times Jw_b$.

Lors du contrôle, le couple gyroscopique a moins d'influence. Si G_a et $w_b \times Jw_b$ sont supprimés, cela a peu d'effet sur les résultats. Après l'avoir retiré, on obtient la formule suivante.

$$\begin{bmatrix} \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} I_{xx} & 0 & 0 \\ 0 & I_{yy} & 0 \\ 0 & 0 & I_{zz} \end{bmatrix} \begin{bmatrix} \dot{w}_{bx} \\ \dot{w}_{by} \\ \dot{w}_{bz} \end{bmatrix}$$

On peut voir dans l'équation ci-dessus que pour trouver l'accélération angulaire du drone sous le système de coordonnées du fuselage w_b , trois moments (τ_x , τ_y , τ_z) et un moment d'inertie autour des trois axes (I_{xx} , I_{yy} , I_{zz}) sont également nécessaires.

3.3.6 Modèle de performance de commande

Selon la vitesse de rotation, la force de portance du système informatique et le couple τ agissant sur le corps peuvent être calculés. La relation entre la portance f_e et le moment τ et la vitesse de rotation w_{bar} est la suivante. L'unité de vitesse d'entrée est un radian par seconde.

$$\begin{bmatrix} f_b \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} c_T & c_T & c_T & c_T \\ -\frac{\sqrt{2}}{2}dc_T & \frac{\sqrt{2}}{2}dc_T & \frac{\sqrt{2}}{2}dc_T & -\frac{\sqrt{2}}{2}dc_T \\ \frac{\sqrt{2}}{2}dc_T & -\frac{\sqrt{2}}{2}dc_T & \frac{\sqrt{2}}{2}dc_T & -\frac{\sqrt{2}}{2}dc_T \\ c_M & c_M & -c_M & -c_M \end{bmatrix} \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix}$$

En saisissant la vitesse du moteur, la demi-longueur du fuselage, le coefficient de traction de l'hélice et le coefficient de couple de l'hélice, la force de portance du système UAV et le couple agissant sur le fuselage peuvent être déterminés.

3.3.7 Modèle de groupe moteur

Dans le système d'alimentation du drone, le régulateur électronique sert d'unité de commande

du moteur et reçoit le signal de contrôle PWM émis par le contrôleur de vol. L'amplitude du signal PWM est normalisée dans la plage de 0~1, et la taille du cycle de service du signal détermine directement la tension moyenne de sortie du moteur sans balais par l'ESC, puis ajuste la vitesse de rotation réelle du moteur.

Du point de vue de la théorie du contrôle, le processus de réponse dynamique d'un moteur sans balais, de l'entrée de commande PWM à la sortie à vitesse réelle, présente certaines caractéristiques d'inertie et d'hystérésis, qui peuvent être modélisées de manière équivalente comme un système inertiel du premier ordre. Après avoir ignoré les transitoires électromagnétiques complexes, la friction mécanique et les facteurs de perturbation non linéaires à l'intérieur du moteur, le changement dynamique de vitesse du moteur peut être décrit avec précision par des équations différentielles du premier ordre ou des fonctions de transfert du premier ordre afin de caractériser la relation de cartographie dynamique entre le contrôle de la PWM et la vitesse réelle du moteur.

$$\begin{aligned}\varpi_{ss} &= C_R \sigma + \varpi_b \\ \varpi &= \frac{1}{T_m s + 1} \varpi_{ss} \\ &= \frac{1}{T_m s + 1} (C_R \sigma + \varpi_b)\end{aligned}$$

Paramètre	Signification
σ	Commande de gaz normalisée
C_R (PowerUnit_CR)	Coefficient de proportionnalité vitesse-commande
ϖ_b (PowerUnit_wb)	Constante de vitesse de base
ϖ_{ss}	Vitesse de rotation en régime permanent
T_m (PowerUnit_Tm)	Constante de temps de réponse
ϖ	Vitesse de rotation réelle du moteur

3.3.8 Modèle globale de simulation du drone sur simulink

Comme illustré sur la figure, il s'agit du modèle basic du drone construit dans ce travail. En excluant certaines perturbations extérieures, ce modèle peut être considéré comme un modèle idéal et simplifié.

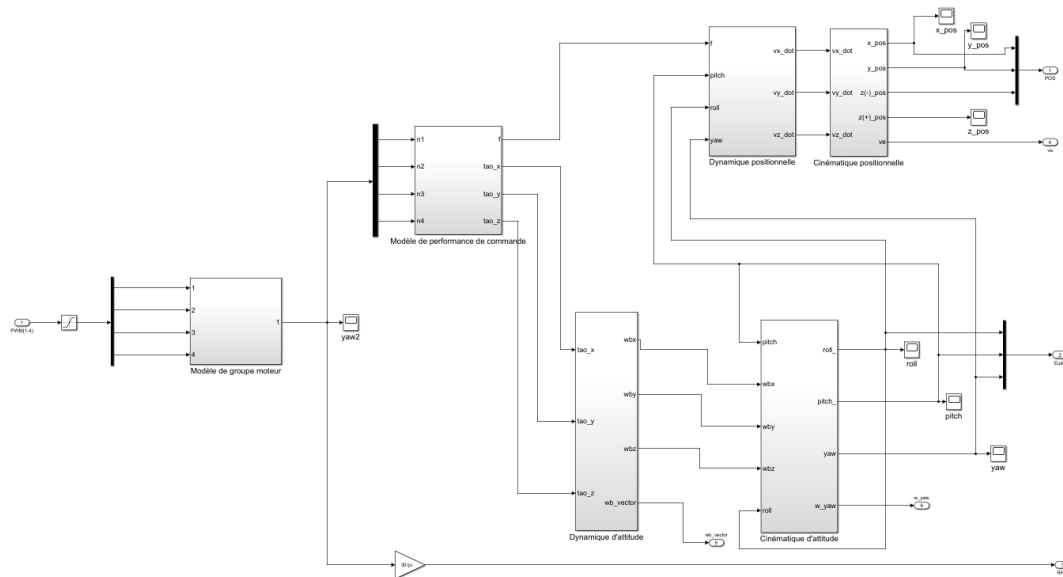


Fig3.3.8 : Modèle globale de simulation du drone sur simulink.

On prend le modèle de l'unité de puissance comme limite : la partie à droite constitue l'ensemble du modèle dynamique et cinématique du drone, tandis que la partie à gauche correspond aux entrées de commande du système.

Le module situé à l'extrême droite du schéma permet l'interaction de données avec le logiciel de simulation FlightGear. Il affiche en temps réel l'attitude et la position du drone dans l'environnement visuel de FlightGear, ce qui permet d'évaluer intuitivement les performances de l'algorithme de commande.

Pour l'utilisation de la cosimulation, le pas de simulation doit être configuré en pas fixe, et sa valeur doit être identique au pas d'envoi du bloc « Send net_fdm Packet to FlightGear » dans le module To FlightGear.

FlightGear requiert la vitesse de rotation du moteur en unité rpm (tour par minute), tandis que la sortie calculée par le modèle de l'unité de puissance s'exprime en rad/s (radian par seconde). Une conversion d'unité est donc nécessaire : il suffit de multiplier la vitesse angulaire obtenue par le coefficient de conversion $\pi/30$.

3.4 Principes de base du contrôle de vol

Le contrôle du drone est divisé en deux parties : le contrôle d'attitude et le contrôle de position.

Atteindre le contrôle de position est l'objectif ultime; pour obtenir un déplacement, le drone doit changer d'attitude afin de générer des forces autres que la direction verticale pour pousser le drone à se déplacer, et l'UAV ne doit pas passer dans une attitude arbitraire, donc l'attitude du drone doit être contrôlée, ce qui explique aussi pourquoi le contrôle d'attitude est la base et le principe du contrôle de position.

3.4.1 Le schéma du contrôleur et du modèle linéarisé

Les drones quadricoptères sont un système typique de sous-propulsion. L'entrée est une force totale f et un moment à trois axes τ , tandis que la sortie a 6, c'est-à-dire 3 degrés de liberté translationnels et 3 degrés de liberté rotationnels. Ainsi, le multirotor ne peut suivre que 4 instructions souhaitées (position désirée et angle de lacet désiré), et les variables restantes (θ_d , ϕ_d) sont calculées par la position désirée et l'angle de lacet attendu.

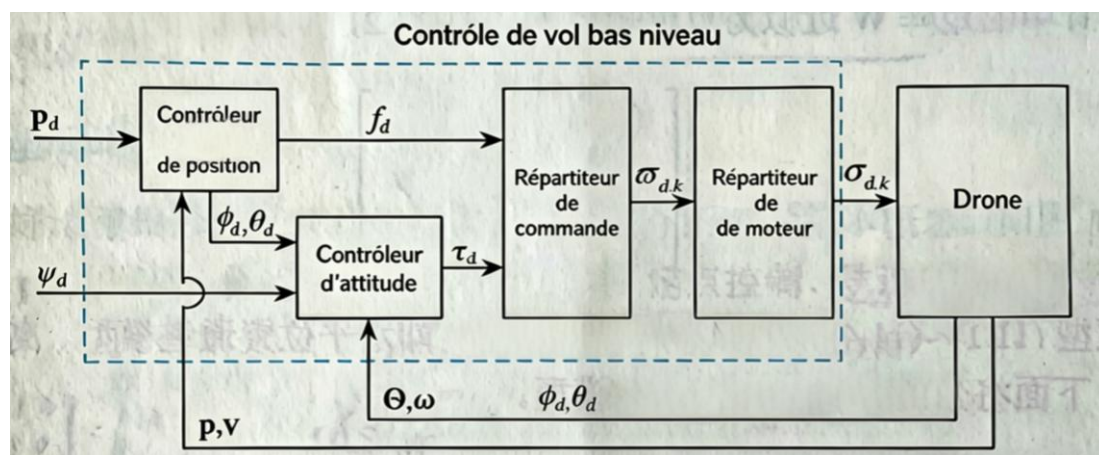


Fig3.4.1 : Structure en boucle fermée du drone sous-jacent au contrôle de vol.

Dans le schéma global du contrôleur, les entrées données au système sont la position désirée P_d et l'angle de lacet désiré ψ_d . Le contrôleur de position calcule, à partir de la position désirée P_d , de la position réelle P_{eet} de la vitesse réelle V_e dans le repère spatial, la poussée désirée f_d , ainsi que les angles de roulis et de tangage désirés ϕ_d et θ_d . Ensuite, les angles d'attitude désirés θ_d , ϕ_d , ψ_d , les angles d'Euler réels Θ et la vitesse angulaire réelle du corps ω_b sont injectés dans le contrôleur d'attitude pour obtenir les moments triaxiaux désirés τ_d .

Ensuite, le modèle de répartition de commande calcule la vitesse de rotation désirée de chaque moteur $\omega_{d,k}$ à partir de la poussée désirée f_d et des moments triaxiaux désirés τ_d . Ce module a pour rôle de déterminer les vitesses de rotation correspondantes à partir de la poussée et des moments, ce qui est l'opération inverse du modèle de performance de commande présenté précédemment, qui calcule la poussée et les moments à partir de la vitesse de rotation.

Enfin, le contrôleur moteur génère la commande de gaz désirée $\sigma_{d,k}$ à partir de la vitesse de rotation désirée $\omega_{d,k}$, ce qui constitue également un processus inverse par rapport au modèle de

groupe moteur.

En résumé, le contrôleur a pour objectif de générer les commandes de gaz correspondantes au moyen du contrôleur de position et du contrôleur d'attitude.

Par ailleurs, comme on peut le constater à partir du résumé de la section 2.3.3, les équations du quadrirotor contiennent des fonctions trigonométriques, ce qui confère au modèle un caractère non linéaire. Pour la conception du contrôleur, il est nécessaire de linéariser le modèle autour d'un point d'équilibre. Le vol stationnaire du drone constitue un tel point d'équilibre, car tous les états du système y sont constants. Dans cette configuration, les angles d'Euler (à l'exception de l'angle de lacet ψ) sont proches de zéro, ce qui permet d'appliquer l'approximation des petits angles.

3.4.2 Construction du modèle de contrôle basée sur Simulink

La section présente la simulation préliminaire du modèle de contrôle du drone, réalisée avec des données hypothétiques. Son objectif est d'apprendre et de valider les principes du contrôle de vol et de la stabilisation du drone, afin de jeter les bases des travaux de recherche ultérieurs.

3.4.2.1 Contrôle du distributeur et du contrôleur moteur

Lorsque Simulink a été effectivement construit, un module de « commande mixte » était utilisé à la place du distributeur de commande et du répartiteur moteur dans le schéma en blocs.

Entrée : valeur PWM (0-1) pour la force de traction désirée f_d et le couple désiré τ

Sortie : Instruction de vitesse (1000-2000) pour quatre moteurs

Le contenu de ce module est présenté ci-dessous.

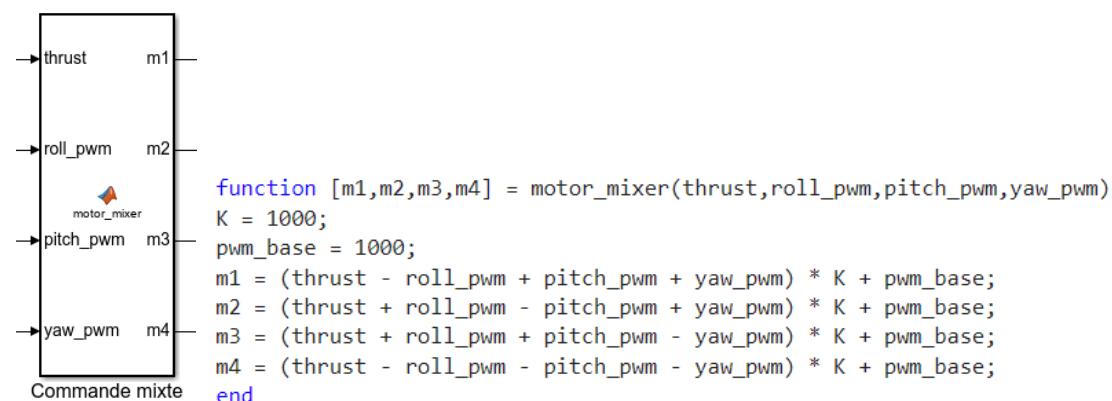


Fig3.4.2.1 : Contrôle du distributeur et du contrôleur moteur sur Simulink.

Comme l'entrée du contrôle mixte est comprise entre 0 et 1 et la sortie entre 1000 et 2000, une amplification est effectuée en premier, puis 1000 est ajoutée.

Le numéro de $\pm$ devant chaque élément est lié à l'ordre de rotation de la crémaillère et du moteur.

Pour le moteur 1, plus la manette des gaz est grande, plus la vitesse est élevée, donc la valeur PWM du moteur 1 doit être augmentée, de sorte que le symbole devant « poussée » est positif ; En même temps, selon la règle de la spirale de droite, la rotation antihoraire autour de l'axe des x est dans le sens positif, si on veut que le corps tourne d'un angle ϕ positif autour de l'axe des x, la valeur de PWM du moteur 1 doit être réduite, de sorte que le symbole devant « roll_pwm » est négatif ; De la même manière, si on veut que le corps tourne un angle θ positif autour de l'axe y, vous devez augmenter la valeur de PWM du moteur 1, de sorte que le symbole devant « pitch_pwm » soit positif ; Si on veut que le corps tourne à un angle de ψ positif autour de l'axe z, on peut augmenter la valeur de PWM du moteur 1, de sorte que le symbole devant « yaw_pwm » soit positif.

3.4.2.2 Conception du contrôleur d'attitude

Le contrôleur d'attitude ici est contrôlé par un PID série.

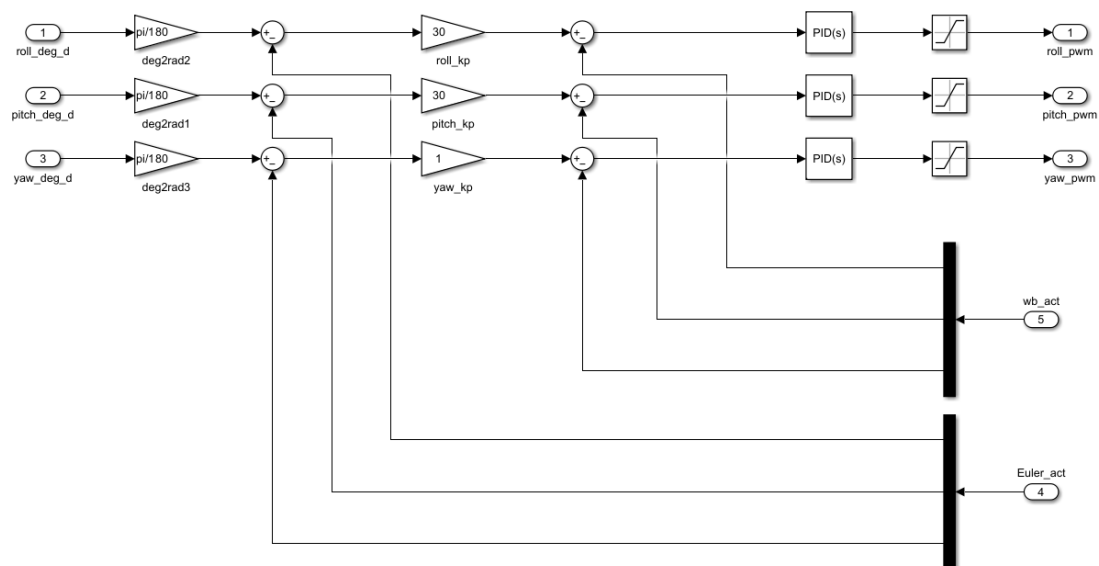


Fig3.4.2.2-1 : Le contrôleur d'attitude sur Simulink.

Lorsqu'il y a une erreur de position, on peut la réduire en ajustant la vitesse. Lorsque l'erreur de position est grande, la vitesse attendue est également grande, et lorsque l'erreur de position est faible, la vitesse attendue est également faible. Nous atteignons le but de contrôler la position en contrôlant la vitesse désirée.

Dans la boucle angulaire, l'erreur angulaire des trois angles d'Euler est en entrée, et la sortie est la vitesse angulaire attendue sous le système machine.

Dans l'anneau de vitesse angulaire, l'entrée est la vitesse angulaire attendue, et la sortie est le couple triaxial attendu.

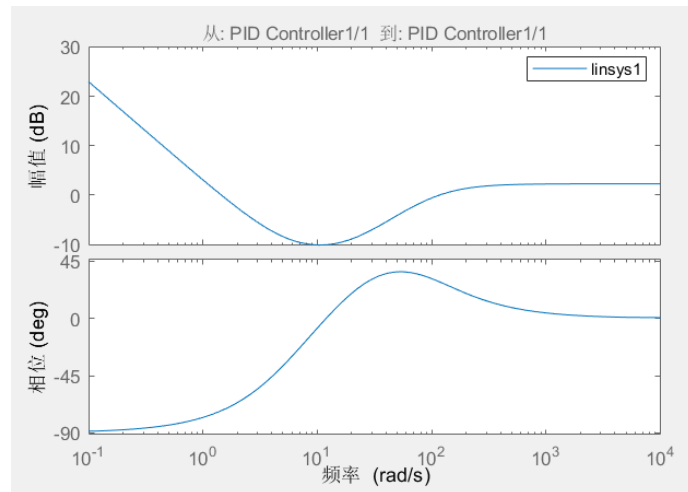


Fig3.4.2.2-2 : Schéma de Bode du contrôle de l'angle de roulis dans le contrôleur de position.

Les paramètres de contrôle PID que j'ai conçus sont stables, présentent un gain élevé en basse fréquence et permettent d'éliminer les erreurs en régime permanent, mais la marge de phase n'est pas particulièrement grande. Et la bande passante n'est pas très élevée, et la réponse aux instructions qui changent rapidement sera lente.

3.4.2.3 Conception des contrôleurs de position

La conception du contrôleur de position est similaire à celle du contrôleur d'attitude, et il adopte également le contrôle PID série, la bague extérieure contrôle la position, seule la commande P, et la bague intérieure contrôle la vitesse, qui adopte le contrôle PID. On utilise la position position-réelle attendue comme entrée et la sortie finale est de trois angles d'Euler.

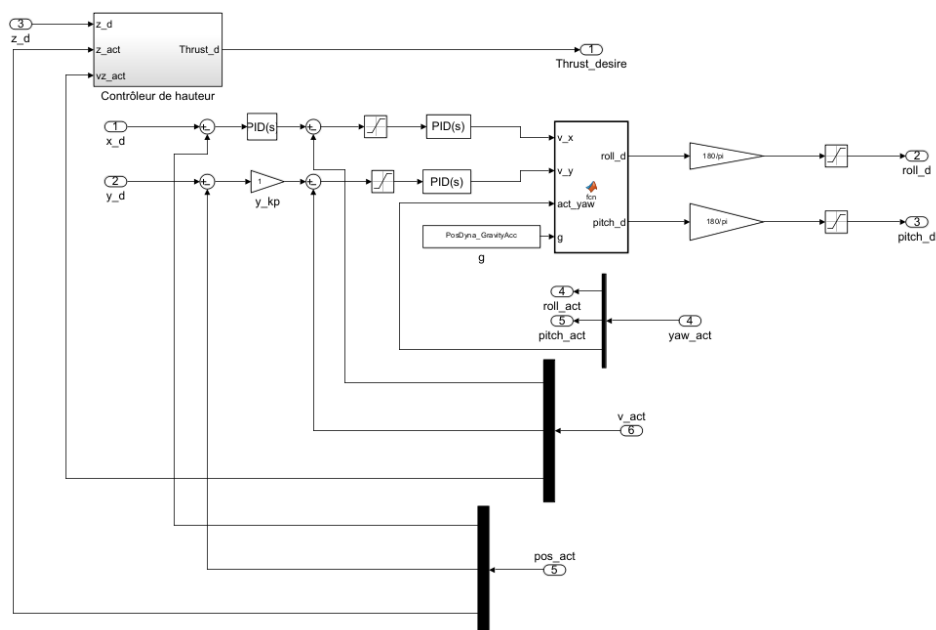


Fig3.4.2.3 : La conception du contrôleur de position sur Simulink.

Puisque les équations dynamiques dans les directions x et y sont liées aux trois angles eulériens, et que l'axe z ne contient pas les angles d'Euler, nous pouvons les concevoir séparément, avec un ensemble de contrôleurs dans les directions x et y, et un contrôleur séparé dans la direction z. Le contrôleur dans les directions X et Y est également appelé contrôleur de canal de position horizontale, et celui dans la direction z est aussi appelé contrôleur de canal de hauteur.

L'entrée du canal horizontal est la position désirée dans les directions x et y sous le système de coordonnées spatiales, et la sortie est l'angle de roulis désiré φ et l'angle de tangage θ .

Le canal d'altitude entre la hauteur désirée, l'altitude réelle et la vitesse réelle de l'axe z, et fournit la force de traction attendue du drone.

Ici, j'ai ajouté un module limiteur à l'erreur de vitesse de l'anneau intérieur, sinon l'avion perdait le contrôle. En même temps, un module de limitation est également ajouté à l'angle attendu de la sortie finale afin d'éviter que l'angle souhaité ne soit trop grand et ne perturbe l'équilibre.

3.4.3 Simulation de contrôle de drone basée sur Simulink

Combiné à la modélisation système de la partie précédente, nous construisons un modèle de simulation de contrôle en boucle fermée de UAV multi-rotors sur Simulink, et analysons les résultats des expériences de suivi de trajectoire circulaire.

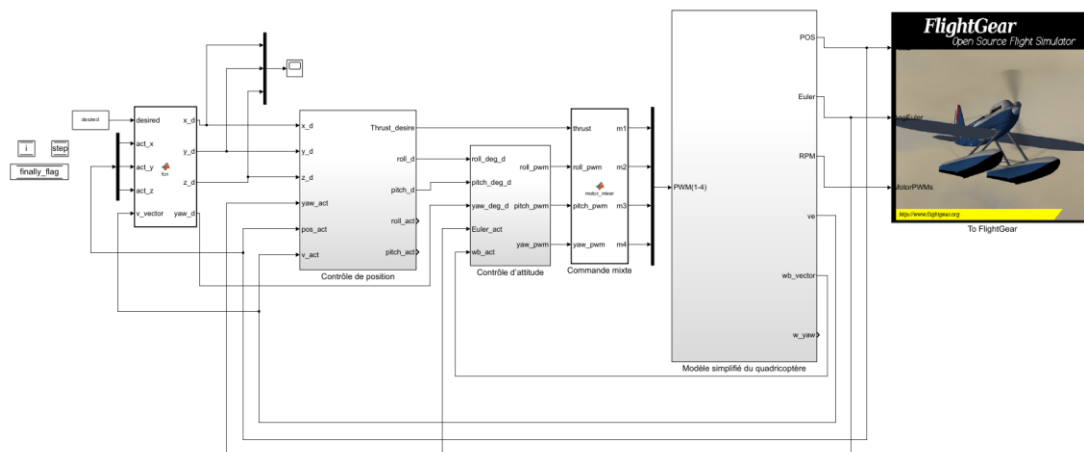


Fig3.4.3-1 : Modèle de simulation de contrôle en boucle fermée de drone sur Simulink.

Comme montré sur la figure ci-dessus, le modèle adopte une structure de contrôle en cascade dans son ensemble : le côté gauche est le module de planification de trajectoire, qui génère une séquence de commandes de position selon la trajectoire prédéfinie; L'anneau extérieur est un contrôleur de position, qui affiche la force de traction attendue et la commande d'angle d'attitude selon la position attendue et l'erreur de position réelle. L'anneau intérieur est le contrôleur d'attitude, et le moment de contrôle est calculé en fonction de l'erreur d'attitude. Le module de distribution de contrôle convertit la commande de couple en signal PWM de chaque moteur et

l'introduit dans le modèle simplifié du drone. Les informations d'état (position, attitude, vitesse angulaire) fournies par le modèle sont renvoyées au contrôleur, et les données de vitesse de rotation et d'attitude sont envoyées à FlightGear pour une visualisation 3D.

La figure suivante montre la séquence de position plane de la simulation de suivi de trajectoire circulaire, qui reflète intuitivement la différence de caractéristiques de mouvement selon les segments de trajectoire.

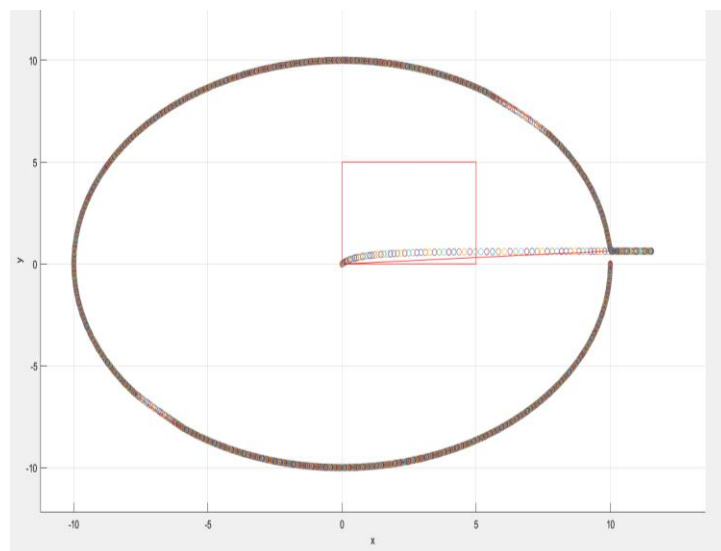


Fig3.4.3-2 : Les résultats de la simulation de trajectoire du modèle drone basé sur MATLAB et Simulink.

Dans le processus de déplacement linéaire, les points de position du drone sont faiblement répartis, indiquant que sa vitesse de vol est rapide. Il y a un léger dépassement à la fin de la trajectoire, après quoi le drone revient sur la trajectoire prédéterminée et continue d'effectuer le suivi circulaire. Après être entré dans la section de trajectoire circulaire, la distribution des points de position est nettement plus dense et la vitesse de déplacement est fortement réduite. Les causes de ce phénomène peuvent se résumer en deux aspects : premièrement, le suivi de trajectoire circulaire exige plus de précision du contrôle d'attitude et de volume de calcul, et le contrôleur doit ajuster continuellement l'angle de roulis et l'angle de tangage pour maintenir l'accélération centripète, la charge de calcul étant supérieure à celle de la section droite uniforme. Deuxièmement, afin d'assurer la précision du suivi de trajectoire pendant le processus de virage, l'algorithme de contrôle introduit des mécanismes de limitation et d'amortissement de vitesse pour supprimer les dépassements et éviter l'oscillation, réduisant ainsi la vitesse de vol.

En général, les résultats de la simulation vérifient la traçabilité des UAV aux trajectoires circulaires, et exposent également la robustesse de l'algorithme à l'étape de transition de trajectoire. À l'avenir, le phénomène de dépassement des segments droits pourra être amélioré en optimisant l'algorithme de planification de trajectoire, en ajustant les paramètres de contrôle ou en introduisant une compensation d'avance d'avance, et l'efficacité du suivi des segments de

trajectoire circulaire pourra être améliorée.

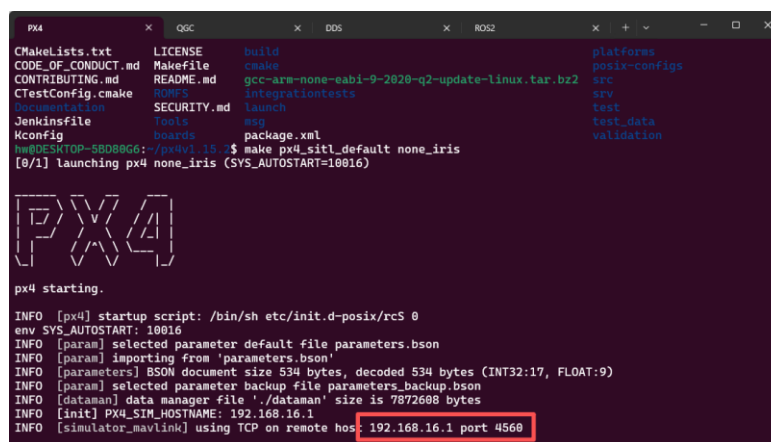
4. Mise en place de la plateforme de simulation et communication entre modules

4.1 Présentation brief des outils de simulation

La plateforme de simulation du projet est déployée en combinant Windows natif et WSL (Ubuntu), avec quatre outils clés intégrés : PX4, AirSim, ROS2 et QGroundControl (QGC). Chaque outil est déployé dans un environnement spécifique, assume une fonction distincte et complémentaire, formant la base de la plateforme de simulation et fournissant un support solide pour les communications entre modules et les fonctions de R&S. Tous les déploiements et configurations des outils sont basés sur mes opérations pratiques.

4.1.1 PX4 Autopilot

Déployé dans WSL (Ubuntu) sous Windows, PX4 Autopilot est le cœur de contrôle de vol du drone, responsable de la stabilisation du vol, de la planification de trajectoire et de la réception des commandes. Il prend en charge les protocoles de communication MAVLink et DDS, permettant l'interaction fluide avec QGC et ROS2.



```
px4
LICENSE
build
platforms
CODE_OF_CONDUCT.md Makefile cmake posix-configs
CONTRIBUTING.md README.md gcc-arm-none-eabi-9-2020-q2-update-linux.tar.bz2 src
CTestConfig.cmake scripts integrationtests ssv
documentation SECURITY.md launch test
Jenkinsfile Tools msg test_data
Kconfig boards package.xml validation
hw@DESKTOP-SBD88G6:~/px4v1.15.1$ make px4_sitl_default none_iris
[0/1] launching px4 none_iris (SYS_AUTOSTART=10016)

PX4

px4 starting.
INFO [px4] startup script: /bin/sh etc/init.d-posix/rcS 0
env SYS_AUTOSTART: 10016
INFO [param] selected parameter default file parameters.bson
INFO [param] importing from 'parameters.bson'
INFO [parameters] BSON document size 534 bytes, decoded 534 bytes (INT32:17, FLOAT:9)
INFO [param] selected parameter backup file parameters.backup.bson
INFO [dataman] data manager file './dataman' size is 7872608 bytes
INFO [init] PX4_SIM_HOSTNAME: 192.168.16.1
INFO [simulator_mavlink] using TCP on remote host: 192.168.16.1 port 4560
```

Fig4.1.1-1 : Capture d'écran de l'interface de démarrage réussi de PX4 dans le terminal WSL, montrant l'état de fonctionnement normal de PX4.

Étant donné que nous n'avons pas démarré Airsim pour le moment, la connexion réussie n'est pas affichée.

4.1.2 AirSim

Installé directement sur Windows natif, AirSim fournit un environnement de simulation virtuel réaliste pour le drone, incluant le modèle de drone, les capteurs simulés (caméra, LiDAR) et les scènes de R&S. Son fichier de configuration (settings.json) enregistre l'IP de WSL et le port de

communication, garantissant la connexion avec les modules déployés dans WSL. Il communique avec ROS2 via son propre protocole API Server.

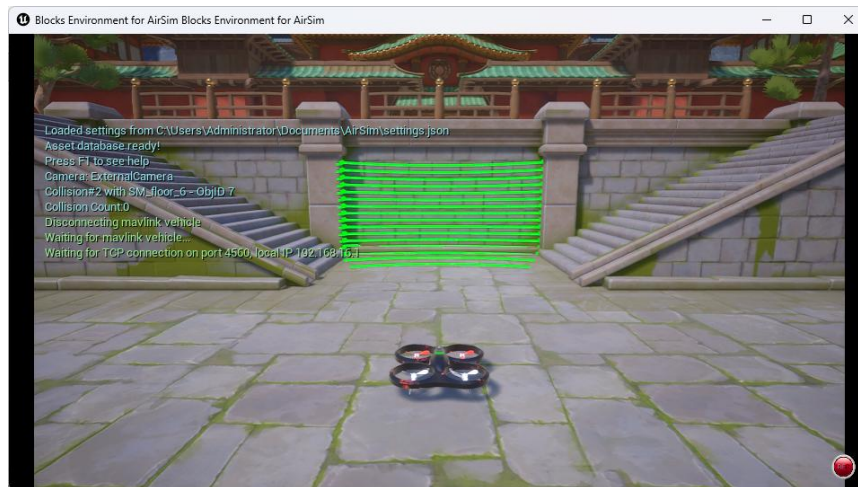


Fig4.1.2-1 : Capture d'écran de l'interface de la scène de simulation après le démarrage d'AirSim sur Windows natif, montrant le drone virtuel et l'environnement.

4.1.3 ROS2

Déployé dans WSL (Ubuntu) sous Windows, ROS2 joue le rôle de middleware de communication centralisé, assurant la transmission de données entre PX4, AirSim et les modules de traitement d'image. Il utilise le protocole DDS pour interagir avec PX4 et l'API Server propre à AirSim pour récupérer les données de la simulation.

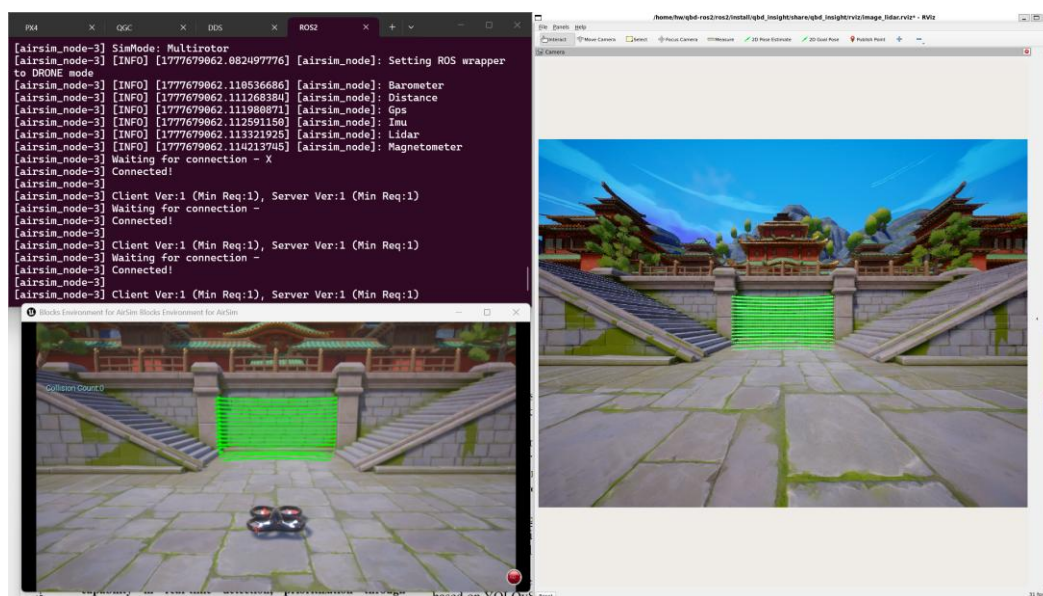


Fig4.1.3-1 : Capture d'écran du démarrage réussi des nœuds ROS2 dans le terminal WSL, montrant l'état de fonctionnement des nœuds concernés.

4.1.4 QGroundControl (QGC)

Déployé à la fois sur Windows natif et dans WSL (Ubuntu) : QGC sous WSL (Ubuntu) communique avec PX4 via le protocole MAVLink UDP sur le port 18570 ; QGC sur Windows natif communique avec PX4 via le protocole UDP sur le port 18570 en utilisant l'IP de WSL, réalisant la supervision et la configuration du drone dans différents environnements de travail.

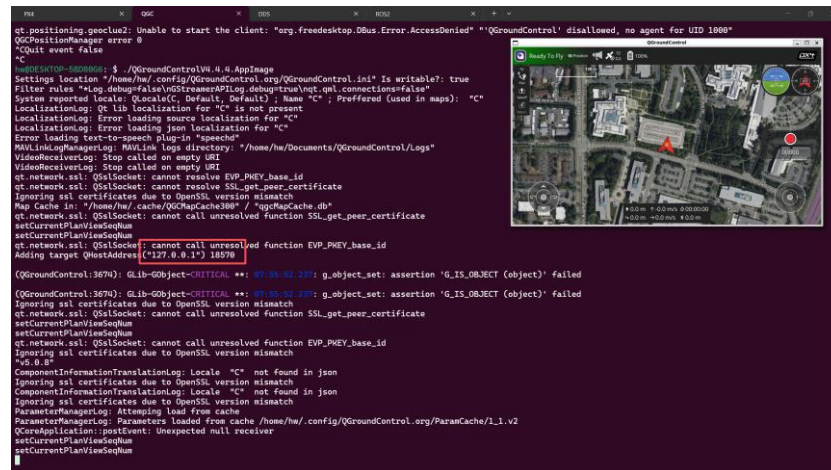


Fig4.1.4-1 : Capture d'écran de l'interface de démarrage de QGC dans WSL, montrant l'état de fonctionnement normal de QGC dans l'environnement WSL.

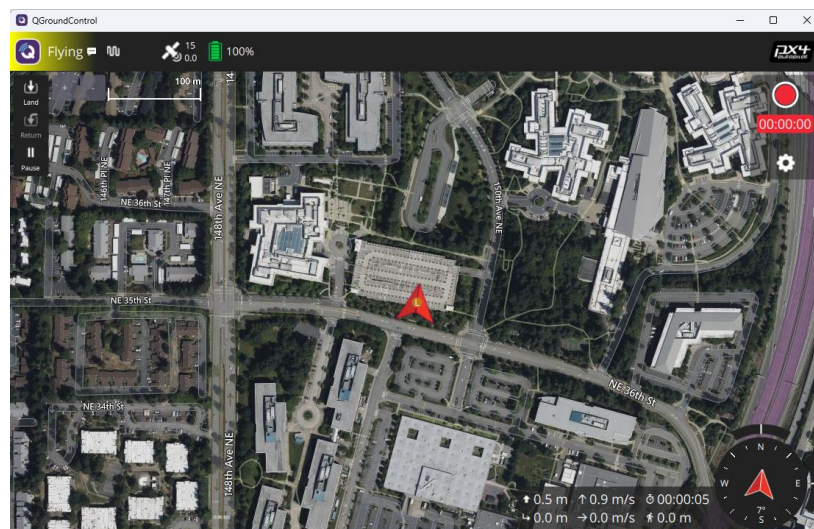


Fig4.1.4-2 : Capture d'écran de l'interface de démarrage de QGC sur Windows natif, montrant l'état de fonctionnement normal de QGC dans l'environnement Windows.

4.2 Détails de la communication entre modules

La communication entre les modules est le cœur de la plateforme de simulation, assurant la coordination fluide entre les outils déployés sur Windows natif et WSL (Ubuntu). Tous les modes de communication disposent de paramètres de configuration et de protocoles spécifiques,

garantissant la fiabilité et la fluidité des échanges .

Il possède des paramètres de configuration et des détails de protocole spécifiques pour garantir la fiabilité et la fluidité de la communication, comme suit :

1. Communication PX4 ↔ ROS2 (tous deux dans WSL) : Utilise le protocole DDS, réalise la transmission bidirectionnelle des données de vol (vitesse, hauteur, position) et des commandes de contrôle, avec des captures d'écran de la connexion réussie disponibles.

```

$ MicroXCAgent udp -p 8888
[177500010 000001] info | MicroXCAgent udp | info | set_verbose_level | port: 8888
[177500010 000001] info | MicroXCAgent udp | info | set_verbose_level | verbose_level: 4
[177500010 000001] info | MicroXCAgent udp | info | create_client | client_key: 0x00000001, session_id: 0x01
[177500010 000001] info | MicroXCAgent udp | info | establish_session | client_key: 0x00000001, address: 127.0.0.1:5825
[177500010 000001] info | MicroXCAgent udp | info | create_participant | participant_key: 0x00000001, participant_id: 0x001(1)
[177500010 000001] info | MicroXCAgent udp | info | create_topic | client_key: 0x00000001, topic_id: 0x000(2), participant_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_subscriber | client_key: 0x00000001, subscriber_id: 0x000(4), participant_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_datareader | client_key: 0x00000001, datareader_id: 0x000(6), subscriber_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_topic | client_key: 0x00000001, topic_id: 0x001(2), participant_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_subscriber | client_key: 0x00000001, subscriber_id: 0x001(4), participant_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_datareader | client_key: 0x00000001, datareader_id: 0x001(6), subscriber_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_topic | client_key: 0x00000001, topic_id: 0x002(2), participant_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_subscriber | client_key: 0x00000001, subscriber_id: 0x002(4), participant_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_datareader | client_key: 0x00000001, datareader_id: 0x002(6), subscriber_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_topic | client_key: 0x00000001, topic_id: 0x003(2), participant_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_subscriber | client_key: 0x00000001, subscriber_id: 0x003(4), participant_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_datareader | client_key: 0x00000001, datareader_id: 0x003(6), subscriber_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_topic | client_key: 0x00000001, topic_id: 0x004(2), participant_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_subscriber | client_key: 0x00000001, subscriber_id: 0x004(4), participant_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_datareader | client_key: 0x00000001, datareader_id: 0x004(6), subscriber_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_topic | client_key: 0x00000001, topic_id: 0x005(2), participant_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_subscriber | client_key: 0x00000001, subscriber_id: 0x005(4), participant_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_datareader | client_key: 0x00000001, datareader_id: 0x005(6), subscriber_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_topic | client_key: 0x00000001, topic_id: 0x006(2), participant_id: 0x0
[177500010 000001] info | MicroXCAgent udp | info | create_subscriber | client_key: 0x00000001, subscriber_id: 0x006(4), participant_id: 0x0

```

Fig4.2-1 : Capture d'écran de la réception et de l'envoi de sujets DDS dans le terminal WSL, montrant la transmission normale des données de vol sans erreur.

2. Communication PX4 (WSL) ↔ QGC (WSL) : Utilise le protocole MAVLink, responsable de la configuration des paramètres de PX4 et de la supervision en temps réel du vol .

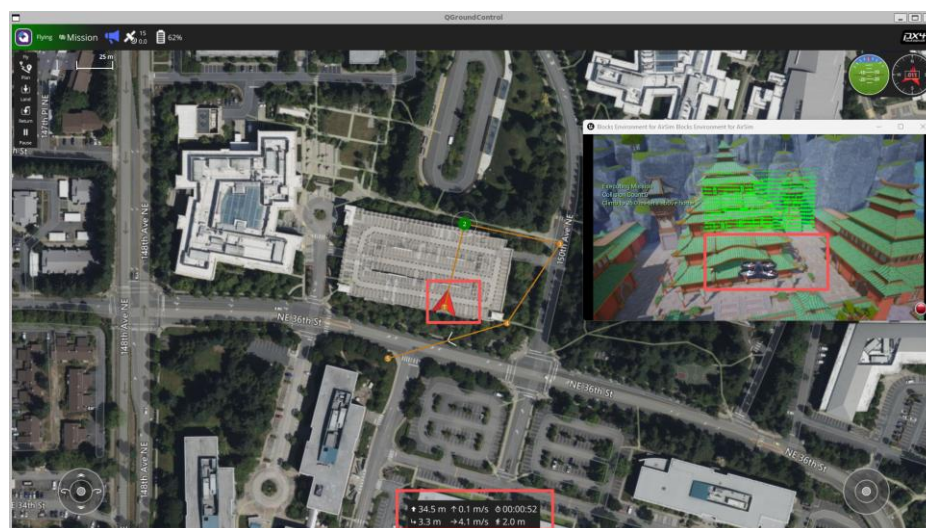


Fig4.2-2 : Capture d'écran de l'interface de connexion réussie entre QGC et PX4 dans WSL, montrant que les données de vol (position, vitesse) se actualisent en temps réel.

3. Communication PX4 (WSL) ↔ QGC (Windows natif) : Utilise le protocole UDP, avec un port inutilisé configuré manuellement pour assurer la communication sans conflit entre les deux

environnements.

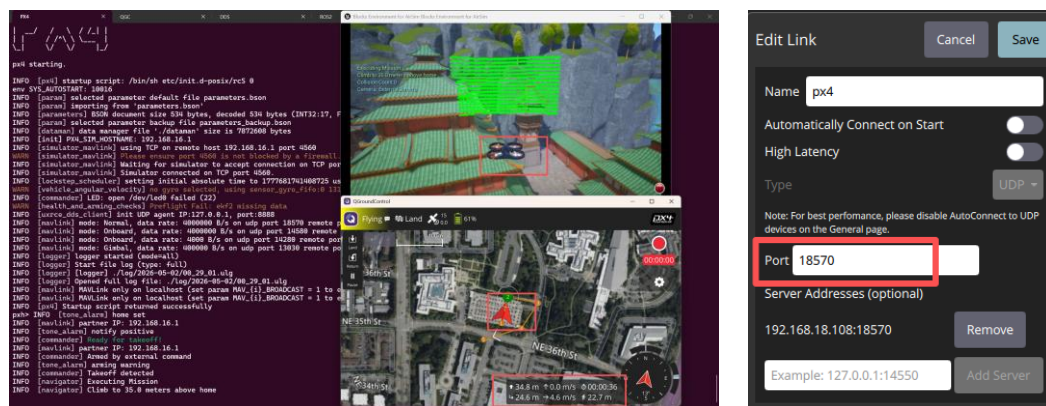


Fig4.2-3 : Capture d'écran de l'interface de configuration du port UDP de QGC sur Windows natif, indiquant la configuration du port 18570 et de l'IP de WSL, et affichant l'état de connexion réussie.

4. Communication AirSim (Windows natif) ↔ ROS2 (WSL) : Utilise le protocole API Server d'AirSim, réalise la synchronisation des données de l'environnement simulé et des capteurs .

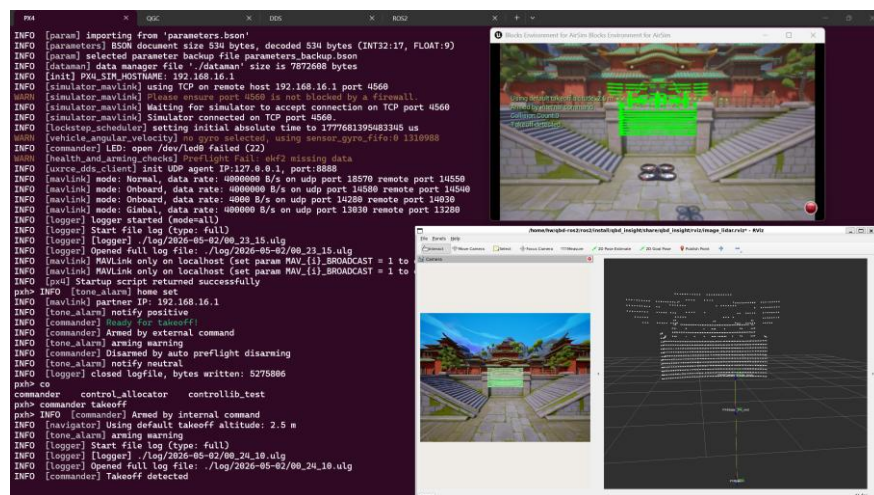


Fig4.2-4 : Capture d'écran du terminal ROS2 recevant les données des capteurs AirSim (telles que l'image de la caméra, le nuage de points LiDAR), montrant la synchronisation normale des données.

5. Configuration d'AirSim : Le fichier settings.json d'AirSim enregistre l'IP de WSL et le port de communication, qui est la clé pour assurer la connexion entre AirSim et les modules de WSL. De plus, AirSim communique avec PX4 via le protocole MAVLink TCP sur le port 4560, réalisant la synchronisation de l'état de vol entre l'environnement simulé et le pilote automatique.

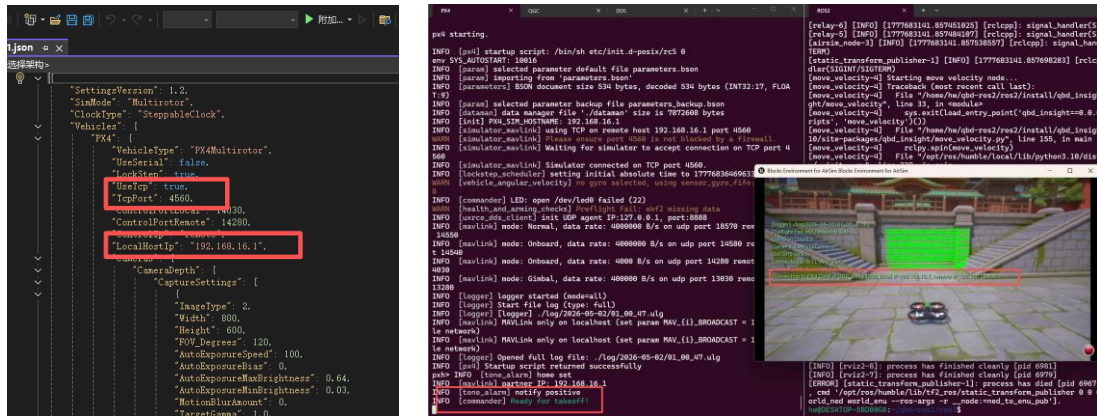


Fig4.2-5 : Capture d'écran du fichier de configuration settings.json d'AirSim, entourant l'IP de WSL, le port de communication et la configuration du port MAVLink TCP 4560.

4.3 Mise en place de l'environnement de simulation et tests de coordination

4.3.1 Processus d'installation et de configuration de l'environnement

Le processus d'installation et de configuration est divisé en 4 étapes clés, en respectant la séparation des environnements Windows natif et WSL (Ubuntu), avec des étapes spécifiques et des paramètres configurés manuellement :

1. Configuration de WSL : Activer WSL sous Windows, installer la distribution Ubuntu, configurer le réseau de WSL pour assurer la communication avec Windows natif, et récupérer l'IP de WSL.

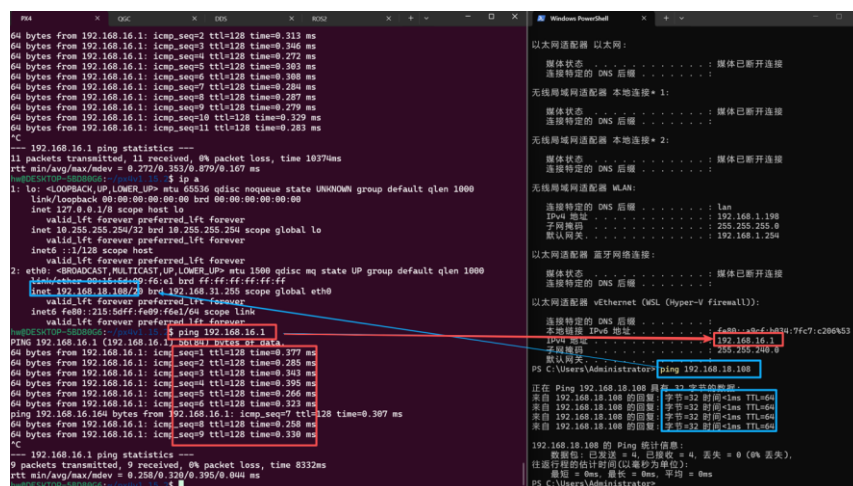


Fig4.3.1-1 : Capture d'écran du test de communication réseau entre WSL et Windows dans le terminal WSL, montrant les résultats du test (ex: commande ping) et prouvant que la communication réseau entre les deux environnements est établie.

2. Installation des outils : Installer PX4 et ROS2 dans WSL (Ubuntu), installer AirSim et QGC sur Windows natif, installer QGC dans WSL (Ubuntu), et configurer les dépendances nécessaires pour chaque outil.

3. Configuration de la communication : Configurer le protocole DDS entre PX4 et ROS2 dans WSL, configurer le protocole MAVLink entre PX4 et QGC (WSL), configurer le protocole UDP et le port inutilisé entre PX4 (WSL) et QGC (Windows natif), modifier le fichier settings.json d'AirSim pour enregistrer l'IP de WSL et le port de communication.

4. Vérification initiale : Lancer chaque outil séparément (AirSim sur Windows, PX4/ROS2/QGC sur WSL), vérifier le fonctionnement normal de chaque module, et confirmer que l'IP et le port sont configurés correctement.

4.3.2 Tests de coordination et de communication entre les modules

Les tests de coordination sont réalisés pour vérifier la fluidité et la fiabilité de la communication entre les modules, avec des résultats de test clairs et des captures d'écran disponibles :

1. Test de communication PX4 ↔ ROS2 (WSL) : Lancer PX4 et ROS2 dans WSL, vérifier via le terminal que le protocole DDS fonctionne normalement, et que les données de vol sont transmises en temps réel sans perte.

2. Test de communication PX4 ↔ QGC (double version) : Lancer QGC sur Windows natif et dans WSL séparément, vérifier que les deux instances de QGC peuvent recevoir les données de PX4 et envoyer des commandes de vol, s'assurer que le protocole MAVLink et UDP fonctionnent correctement.

3. Test de communication AirSim (Windows) ↔ ROS2 (WSL) : Lancer AirSim sur Windows et ROS2 dans WSL, vérifier que les données de l'environnement simulé et des capteurs sont synchronisées via l'API Server .

4. Test de coordination global : Lancer tous les modules simultanément (AirSim sur Windows, PX4/ROS2/QGC sur WSL, QGC sur Windows), vérifier que le drone peut voler dans AirSim, que QGC peut superviser en temps réel, que ROS2 peut transmettre des données normalement, et enregistrer les résultats du test.

4.3.3 Validation de l'efficacité de la simulation de vol de base du drone

La validation de l'efficacité de la simulation de vol de base se concentre sur l'exécution d'une tâche de vol simple du drone, et prouve la fiabilité et l'effectivité de la simulation par les captures d'écran des étapes clés du processus de vol.

Lors du décollage, après avoir lancé tous les modules de simulation, envoyer la commande de

décollage à travers QGC. Les captures d'écran d'AirSim montrent clairement que le drone virtuel s'élève progressivement depuis le sol, sans blocage ni anomalie. L'interface QGC affiche l'état de décollage normal, les données de hauteur commencent à augmenter depuis 0m, prouvant que le décollage du drone est réalisé avec succès dans la simulation.



Fig4.3.3-1 : Capture d'écran de l'interface AirSim pendant le décollage du drone, montrant le drone s'élevant depuis le sol ;

Envoyant la commande de vol linéaire via QGC, le drone vole en ligne droite dans l'environnement AirSim, sans déviation de trajectoire. Les captures d'écran d'AirSim montrent clairement le processus de vol linéaire du drone, et l'interface QGC affiche les données de vitesse stables et la trajectoire normale, prouvant que la simulation peut réaliser le vol linéaire précis du drone.



Fig4.3.3-2 : Capture d'écran de l'interface QGC, montrant les données de vitesse du vol linéaire

Envoyant la commande d'atterrissage via QGC, le drone atterrit progressivement sur le sol dans l'environnement AirSim, sans collision ni anomalie. Les captures d'écran d'AirSim montrent le processus d'atterrissage fluide du drone, et l'interface QGC affiche l'état d'atterrissage réussi, avec les données de hauteur revenant à 0m, prouvant que la simulation peut réaliser l'atterrissage sûr du drone.



Fig4.3.3-3 : Capture d'écran de l'interface AirSim pendant l'atterrissage du drone ;

5. Développement et mise en œuvre des fonctionnalités du système

5.1 Intégration des capteurs simulés

L'intégration des capteurs simulés est la base de l'implémentation des fonctions de R&S du drone, fournissant des données de perception de l'environnement et des cibles pour les étapes ultérieures de détection humaine et de reconstruction 3D. Dans ce projet, nous intégrons trois types de capteurs clés dans l'environnement AirSim : caméra RGB, LiDAR (nuage de points) et caméra de profondeur. Nous réalisons la configuration des capteurs via le fichier de paramètres d'AirSim, achevons la transmission des données des capteurs entre AirSim et ROS2, visualisons les données en temps réel à l'aide de RViz.

5.1.1 Configuration de la caméra, du LiDAR et de la caméra de profondeur dans AirSim

La configuration des capteurs simulés dans AirSim se réalise via le fichier *settings.json*, qui définit le type, les paramètres et la position des capteurs sur le drone virtuel. Nous configurons les trois capteurs selon les paramètres donnés, et vérifions la configuration via l'interface AirSim et le terminal ROS2.

1. Configuration de la caméra RGB (CameraImage) : Nous définissons le type comme « Color », la résolution (800×600), l'angle de vue (120 degrés), et la position (x=0, y=0, z=-1).

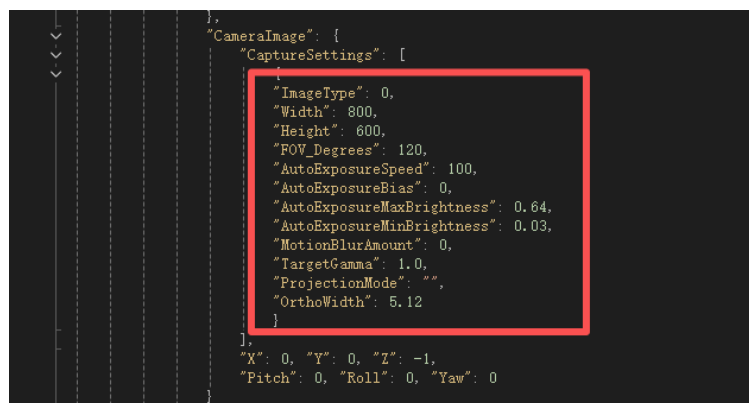
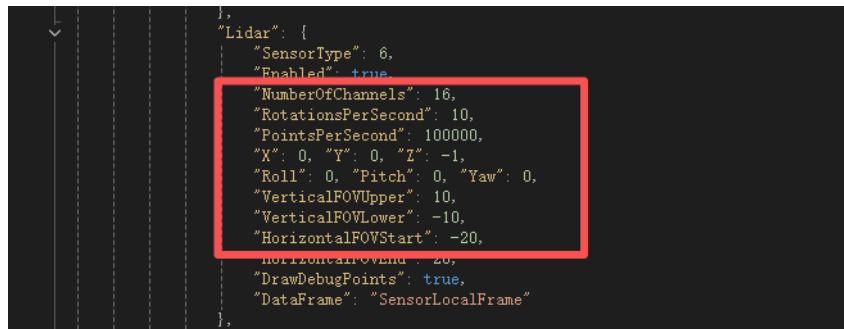


Fig5.1.1-1 : Capture d'écran du segment de configuration de CameraImage dans settings.json

2. Configuration du LiDAR : Nous configurons 16 canaux, 10 rotations par seconde, 100000

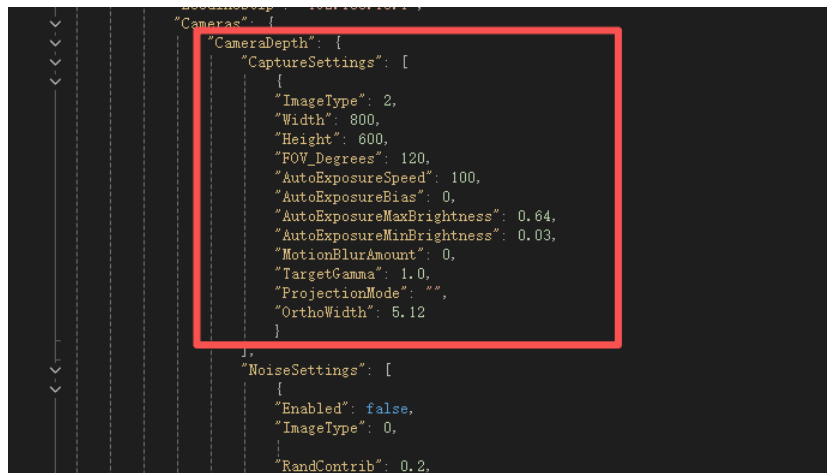
points par seconde, position (x=0, y=0, z=-1), FOV vertical (-10° à 10°) et horizontal (-20° à 20°).

A screenshot of a code editor showing the configuration for a LiDAR sensor in a JSON file. The configuration is enclosed in a red rectangular box. The JSON object is named "Lidar" and contains several key-value pairs for sensor parameters.

```
    "Lidar": {  
      "SensorType": 6,  
      "Enabled": true,  
      "NumberOfChannels": 16,  
      "RotationsPerSecond": 10,  
      "PointsPerSecond": 100000,  
      "X": 0, "Y": 0, "Z": -1,  
      "Roll": 0, "Pitch": 0, "Yaw": 0,  
      "VerticalFOVUpper": 10,  
      "VerticalFOVLower": -10,  
      "HorizontalFOVStart": -20,  
      "HorizontalFOVEnd": 20,  
      "DrawDebugPoints": true,  
      "DataFrame": "SensorLocalFrame"  
    },
```

Fig5.1.1-3 : Capture d'écran du segment de configuration du LiDAR dans settings.json

3. Configuration de la caméra de profondeur (CameraDepth) : Nous définissons la résolution (800×600), l'angle de vue (120 degrés), et la position (x=0, y=0, z=-1).

A screenshot of a code editor showing the configuration for a CameraDepth sensor in a JSON file. The configuration is enclosed in a red rectangular box. The JSON object is named "CameraDepth" and contains a "CaptureSettings" array with various camera parameters.

```
    "CameraDepth": {  
      "CaptureSettings": [  
        {  
          "ImageType": 2,  
          "Width": 800,  
          "Height": 600,  
          "FOV_Degrees": 120,  
          "AutoExposureSpeed": 100,  
          "AutoExposureBias": 0,  
          "AutoExposureMaxBrightness": 0.64,  
          "AutoExposureMinBrightness": 0.03,  
          "MotionBlurAmount": 0,  
          "TargetGamma": 1.0,  
          "ProjectionMode": "",  
          "OrthoWidth": 5.12  
        }  
      ],  
      "NoiseSettings": [  
        {  
          "Enabled": false,  
          "ImageType": 0,  
          "RandContrib": 0.2  
        }  
      ]  
    },
```

Fig5.1.1-5 : Capture d'écran du segment de configuration de CameraDepth dans settings.json

5.1.2 Visualisation des données des capteurs basée sur Rviz

RViz est un outil de visualisation standard et essentiel de ROS2, qui permet d'afficher en temps réel les données des capteurs simulés. Dans cette étape, nous exécutons le script `test_rviz.py` dans l'environnement ROS2 (WSL) pour établir la connexion entre ROS2 et RViz, et réaliser la visualisation des données des capteurs configurés précédemment (caméra RGB, LiDAR, caméra de profondeur). Pour enregistrer les résultats de visualisation, nous capturons deux captures d'écran ciblées, correspondant aux deux scénarios de visualisation demandés.

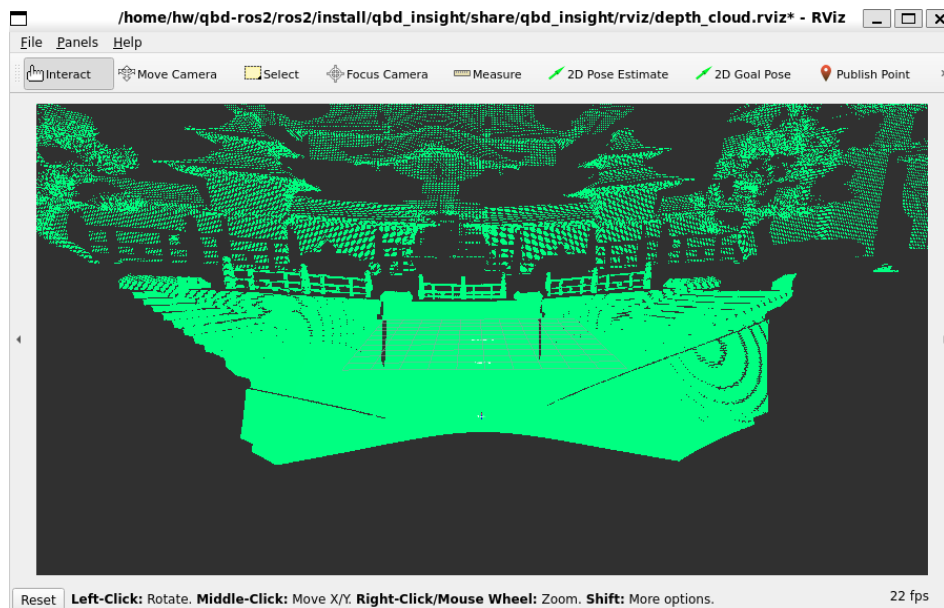


Fig5.1.2-1 : Capture d'écran de l'interface RViz, montrant uniquement l'image de profondeur visualisée

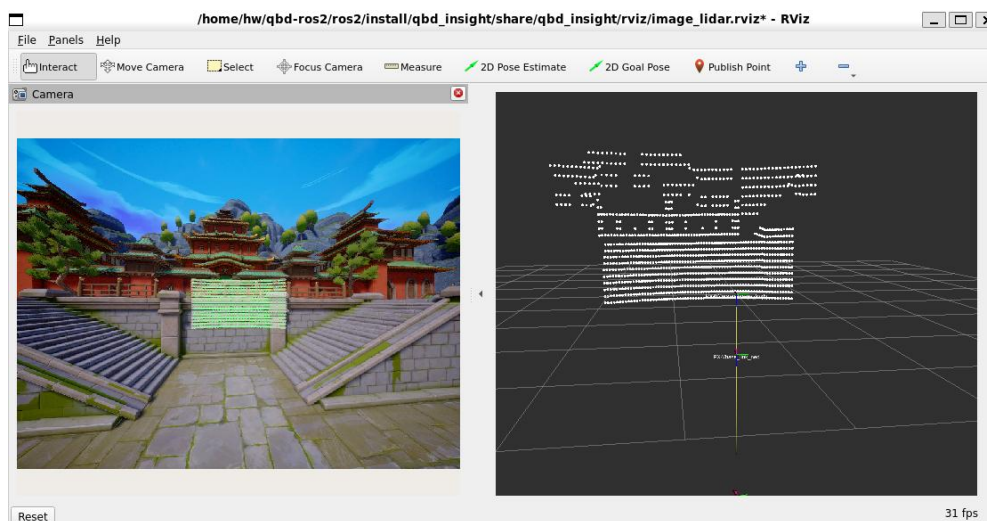


Fig5.1.2-2 : Capture d'écran de l'interface RViz (exécutant test_rviz.py), montrant l'image de la caméra et le nuage de points LiDAR visualisés conjointement.

Les deux captures d'écran enregistrent clairement les effets de visualisation : la première montre uniquement l'image de profondeur transmise par la caméra de profondeur, avec des informations de profondeur de la scène simulée affichées clairement ; la seconde présente l'image RGB de la caméra et le nuage de points du LiDAR visualisés en même temps, permettant de vérifier la cohérence entre les données des différents capteurs. Tous les processus de visualisation sont basés sur l'exécution du script test_rviz.py, garantissant la fiabilité et la réalisabilité pratiques des résultats.

5.2 Détection de la recherche et du sauvetage humain par vision artificielle

5.2.1 Présentation du principe des algorithmes YOLO et OpenCV

Les algorithmes de détection d'objets modernes reposent majoritairement sur les réseaux de neurones convolutifs (CNN). En théorie, il est possible de concevoir et d'entraîner un modèle CNN spécifique pour la détection de personnes, adapté aux besoins du système de recherche et sauvetage. Cependant, l'entraînement d'un modèle personnalisé nécessite une grande quantité de données annotées ainsi qu'un temps de calcul important, ce qui dépasse le cadre de ce projet. De plus, les performances d'un modèle entraîné localement restent fortement dépendantes de la qualité du jeu de données.

Dans ce contexte, nous avons choisi d'utiliser l'algorithme YOLO (You Only Look Once), qui est également basé sur une architecture CNN, mais optimisé pour la détection d'objets en temps réel. YOLO se distingue par sa capacité à effectuer la détection en une seule passe, ce qui permet d'atteindre un bon compromis entre précision et vitesse. Parmi les différentes versions, YOLOv8, proposé par Ultralytics, offre des performances élevées en détection d'objets. Selon les résultats officiels, YOLOv8 atteint une précision élevée (mAP) sur des bases de données standards telles que COCO, avec des performances particulièrement robustes pour la détection de personnes, qui constitue l'une des classes les mieux reconnues. Ainsi, compte tenu de sa robustesse, de sa facilité d'intégration et de ses performances en temps réel, YOLOv8 a été retenu comme solution principale pour la détection de cibles humaines dans ce projet.

5.2.2 Pipeline de détection dans un environnement simulé

Dans cette étude, un environnement simulé simplifié a été construit à l'aide d'un moteur virtuel. Un modèle de personnage humain, issu des ressources de la communauté du moteur, a été intégré dans la scène. Ce personnage peut être contrôlé et déplacé afin de simuler des situations réalistes de recherche et sauvetage. Le drone évolue dans cet environnement et capture un flux vidéo en temps réel. Ce flux est ensuite traité à l'aide de YOLO et de la bibliothèque OpenCV afin de détecter la présence de personnes et de générer des boîtes englobantes autour des cibles identifiées.

Un objectif initial du projet était de permettre au drone de suivre automatiquement la personne détectée. Toutefois, cette fonctionnalité n'a pas pu être pleinement implémentée. En effet, plusieurs difficultés ont été rencontrées, notamment l'absence d'informations précises sur la vitesse du personnage dans l'environnement virtuel, ainsi que la complexité de modéliser une relation dynamique entre la distance drone-cible et la vitesse de suivi. Bien qu'un module de

suivi ait été partiellement implémenté dans le code, les résultats n'ont pas été suffisamment stables pour être démontrés expérimentalement. Ainsi, le système final se concentre sur la détection visuelle en temps réel, qui constitue déjà une base essentielle pour les applications de recherche et sauvetage.

5.2.3 Mise en œuvre de l'effet de détection de piétons dans l'environnement virtuel

Les résultats expérimentaux montrent que le système est capable de détecter la présence de personnes dans l'environnement simulé, avec affichage de boîtes englobantes autour des cibles. Cependant, les performances de détection restent limitées. Le taux de détection observé est d'environ 80 %, avec des cas de non-détection occasionnels. Cette baisse de performance s'explique principalement par le décalage entre les données d'entraînement du modèle YOLO et l'environnement de test.

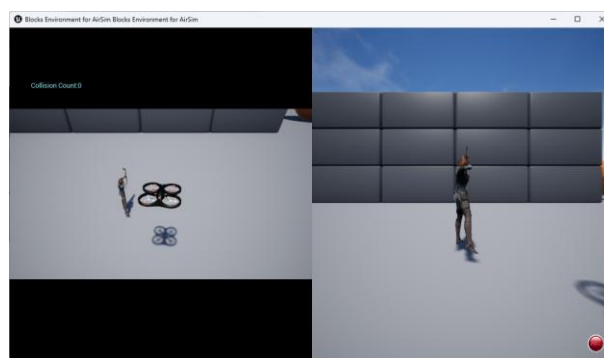


Fig5.2.2-1: Affichage des positions relatives des personnes et des drones

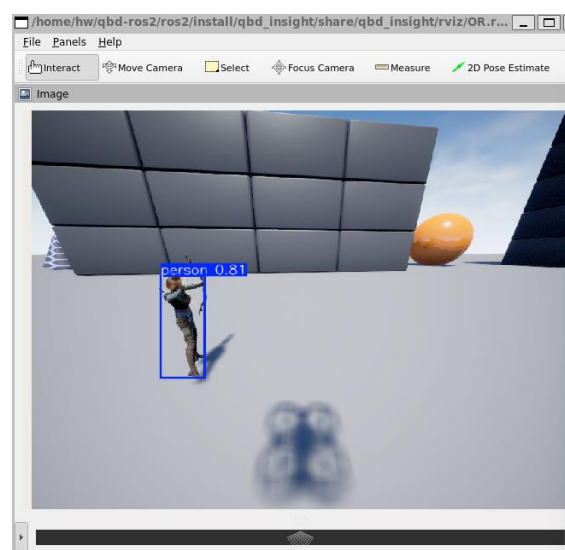


Fig5.2.2-2 : Le drone a identifié et sélectionné des personnes avec un taux de précision de 81 %.

En effet, YOLO est généralement entraîné sur des images de personnes réelles (par exemple dans la base de données COCO), tandis que dans notre simulation, les personnages sont issus d'un moteur de jeu et présentent des caractéristiques visuelles différentes (textures, proportions, animations). Cette différence de domaine entraîne une diminution de la précision du modèle. Malgré ces limitations, les résultats obtenus restent suffisants pour valider la faisabilité de l'approche basée sur YOLO pour la détection de cibles humaines dans un environnement simulé.

5.3 Modélisation de la reconstruction 3D de l'environnement

5.3.1 Principe de la cartographie 3D avec OctoMap

OctoMap est un cadre de cartographie 3D léger, adapté à notre environnement de développement minimaliste. Son principe repose sur la structure octree, qui divise l'espace 3D en voxels et met à jour leur état (occupé, libre, inconnu) selon les données de la caméra de profondeur. Compte tenu des contraintes de performance, nous adoptons une configuration minimale (résolution 200×150 , LockStep, TCP) et OctoMap, avec sa quantification spatiale adaptative, réduit la charge de calcul tout en garantissant une précision relative.

5.3.2 Processus de conversion d'image de profondeur en nuage de points et reconstruction de carte

Le processus de reconstruction 3D est simplifié afin de s'adapter aux ressources limitées du système. Il se compose des étapes suivantes :

1. Acquisition des images de profondeur à une résolution réduite (200×150), avec synchronisation des modules via le mode LockStep ;
2. Prétraitement des données par filtrage médian afin de réduire le bruit de mesure ;
3. Conversion des images de profondeur en nuage de points 3D à l'aide des paramètres intrinsèques de la caméra ;
4. Transmission du nuage de points au système ROS2 via TCP ;
5. Intégration des données dans OctoMap pour mettre à jour les probabilités d'occupation et reconstruire la carte 3D.

5.3.3 Mise en œuvre de l'effet de reconstruction 3D de la scène

simulée

En s'appuyant sur la configuration minimale définie précédemment (résolution 200×150 , synchronisation LockStep et transmission TCP), ainsi que sur le processus de reconstruction décrit en section 4.3.2, la reconstruction 3D de la scène simulée a été réalisée avec succès. Le résultat est visualisé dans RViz sous forme de carte volumétrique issue d'OctoMap. Dans la visualisation obtenue, les différentes couleurs traduisent la distance relative des voxels par rapport à la caméra de profondeur : les zones proches apparaissent en rouge, tandis que les zones plus éloignées sont représentées en bleu. Cette représentation permet d'évaluer qualitativement la structure spatiale de l'environnement reconstruit. L'analyse du résultat montre que la structure globale de la scène est correctement reconstruite, ce qui valide le bon fonctionnement de la chaîne de traitement (acquisition, conversion en nuage de points et intégration dans OctoMap). Toutefois, la précision géométrique reste limitée, avec des contours peu nets et une granularité relativement grossière.

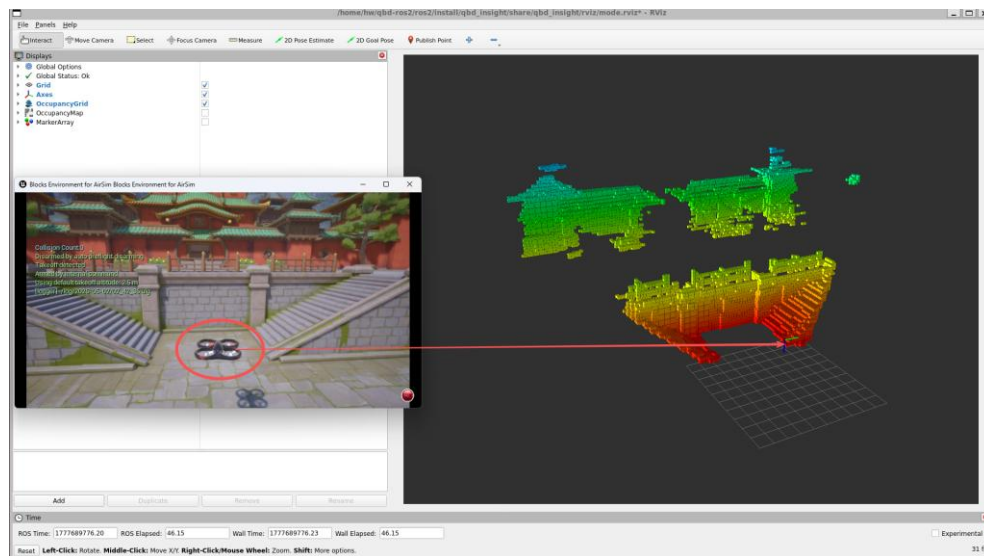


Fig5.3.3-1 : Capture d'écran de la reconstruction 3D de la scène simulée dans RViz, montrant les zones rouges et bleues correspondant à différentes distances

Cette limitation provient principalement du paramètre de résolution de l'OctoMap, fixé à 0,25 m dans le fichier *mode.py*. Ce paramètre définit la taille minimale des voxels ($0,25 \text{ m} \times 0,25 \text{ m} \times 0,25 \text{ m}$), ce qui entraîne une discrétisation spatiale importante et donc une perte de détails dans la reconstruction. Ce choix constitue un compromis entre précision et performance. En effet, une résolution plus fine augmenterait significativement le nombre de voxels à traiter, entraînant une hausse du coût mémoire et du temps de calcul, ce qui n'est pas compatible avec les capacités du système embarqué utilisé (Raspberry Pi) ni avec les

contraintes de démonstration en temps réel. Ainsi, dans un environnement disposant de ressources de calcul plus élevées, il serait possible de réduire la taille des voxels afin d'améliorer la qualité de reconstruction, au prix d'une augmentation de la complexité computationnelle.

6. Résultats expérimentaux et validation des performances du système

6.1 Résultats de simulation du contrôle de vol du drone

Afin d'évaluer la stabilité du drone dans l'environnement simulé, deux types de tests ont été réalisés : un test de vol en position fixe et un test de suivi de trajectoire.

Dans le test de trajectoire, le drone est programmé pour passer successivement par quatre points de passage avant de revenir à sa position initiale. La trajectoire réelle du drone est observée via l'interface QGroundControl (QGC) et comparée à la trajectoire planifiée.

Les résultats montrent que la trajectoire réelle coïncide globalement avec la trajectoire théorique, avec un écart très limité. Aucun phénomène d'oscillation significatif ni de dérive importante n'a été observé au cours du vol. Cela indique que le système de contrôle du drone présente une bonne stabilité et une capacité de suivi de trajectoire satisfaisante dans l'environnement simulé.

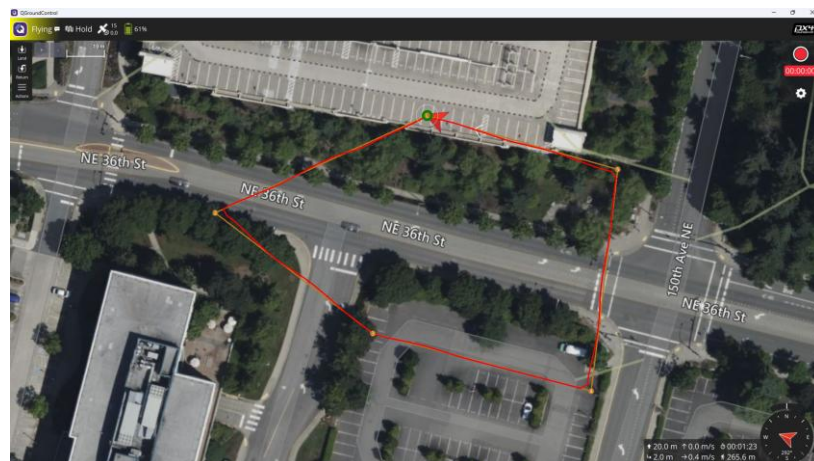


Fig 6.1-1 : Trajectoire réelle du drone lors du suivi de points de passage dans QGroundControl

6.2 Résultats des performances de détection d'objets humains

Les performances du système de détection d'objets humains ont été évaluées à l'aide de deux indicateurs principaux : la précision de détection et la fréquence de traitement (FPS).

Les résultats expérimentaux montrent que le modèle YOLOv8 est capable de détecter efficacement les personnes dans l'environnement simulé, avec un taux de détection d'environ 80 %. Toutefois, des cas de non-détection ou de détection instable apparaissent dans certaines situations, notamment lorsque le personnage est éloigné ou partiellement occulté.

En termes de performance temporelle, le système atteint une fréquence de traitement d'environ 20 FPS, ce qui permet une détection quasi temps réel dans le cadre de la simulation.

Les limitations observées sont principalement dues à l'écart entre les données d'entraînement du modèle (images réelles) et les données de test (personnages virtuels). Cette différence de domaine entraîne une baisse de précision du modèle.

Malgré ces limitations, les résultats obtenus démontrent la faisabilité de l'utilisation de YOLO pour des applications de détection humaine dans un environnement simulé.

6.3 Effet de reconstruction 3D et analyse de précision

Afin d'évaluer l'influence de la résolution sur la qualité de reconstruction 3D, des expériences ont été menées avec différentes tailles de voxels dans OctoMap.

Pour faciliter l'observation, la caméra de profondeur est montée sous le drone, orientée vers le sol avec un angle de vue de 120°. Cette configuration permet de visualiser clairement les différences de qualité de reconstruction.

Lorsque la résolution est fixée à **0,25 m**, la reconstruction est rapide et fluide, mais le modèle reste relativement grossier, avec une perte de détails dans les structures fines.

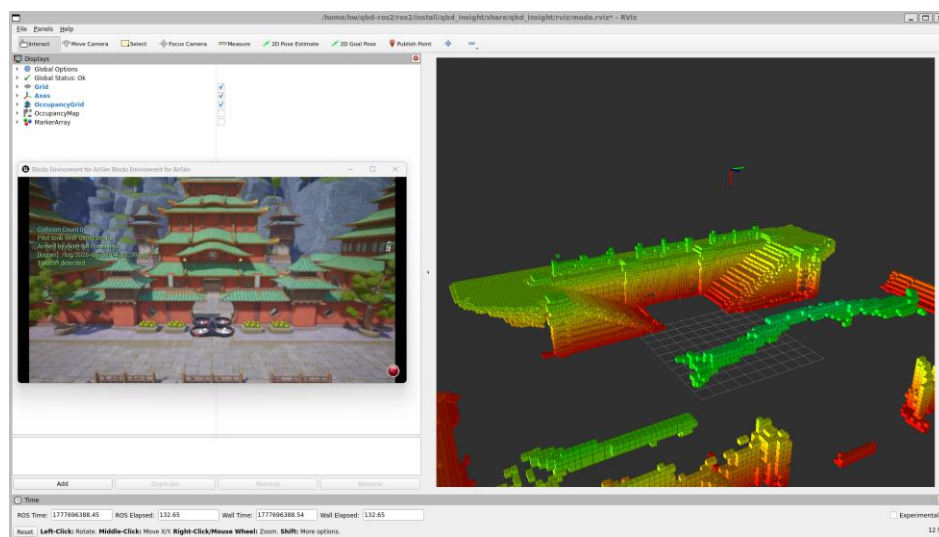


Fig 6.3-1 : Reconstruction 3D de la scène avec une résolution de 0,25 m

En revanche, lorsque la résolution est réduite à **0,1 m**, la qualité de reconstruction s'améliore significativement : les contours deviennent plus précis et les détails mieux représentés. Toutefois, cette amélioration s'accompagne d'une augmentation importante du coût computationnel, entraînant des ralentissements notables du système.

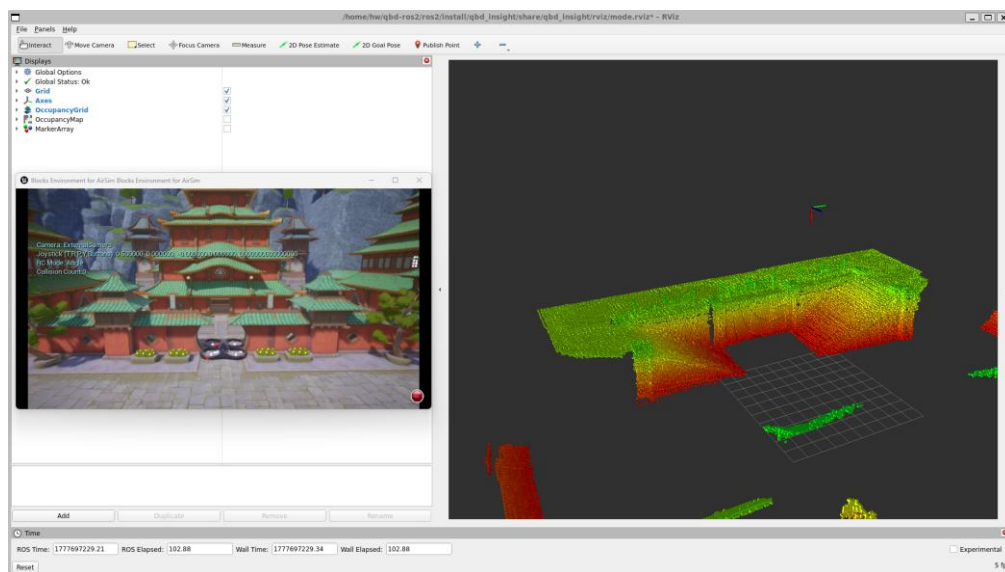


Fig 6.3-2 : Reconstruction 3D de la scène avec une résolution de 0,1 m

Par ailleurs, la vitesse globale de reconstruction dépend également des performances du capteur de profondeur. Dans la configuration actuelle, la portée du capteur est limitée à 25 m. Une augmentation de cette portée permettrait d'accélérer la couverture de l'environnement, sans affecter directement la qualité intrinsèque de la reconstruction. Ainsi, la résolution de 0,25 m a été retenue comme compromis entre précision et performance, adapté aux contraintes du système embarqué.

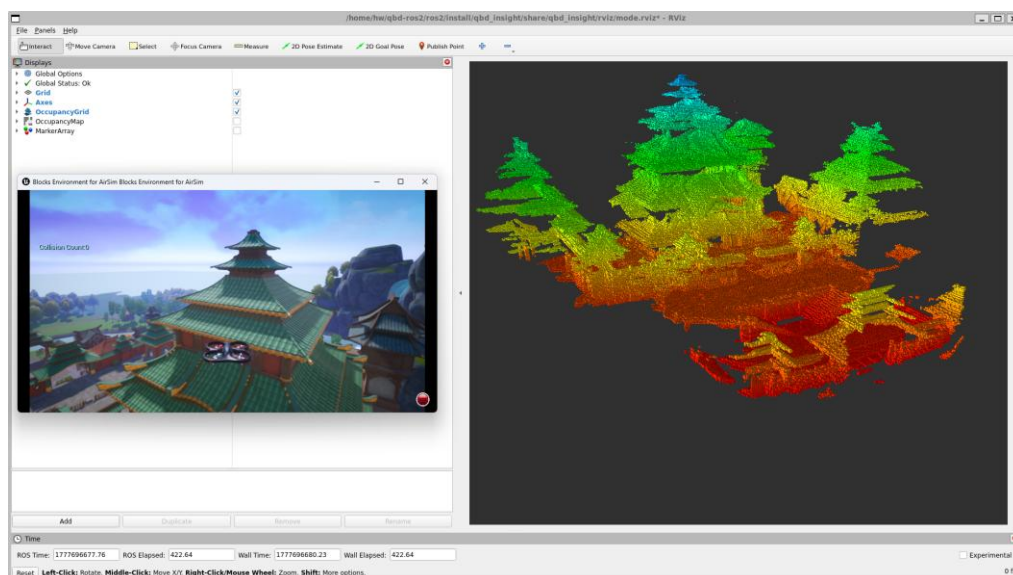


Fig 6.3-3 : Reconstruction 3D globale de l'environnement simulé

7. Conclusion générale

Ce projet avait pour objectif la conception et la simulation d'un système de drone autonome dédié aux missions de recherche et de sauvetage (R&S) dans un environnement virtuel, en intégrant de manière cohérente le contrôle de vol, la vision artificielle, la reconstruction 3D et l'étude des communications 5G. Dans un contexte où les systèmes multi-agents deviennent de plus en plus importants, l'analyse du réseau 5G a constitué une composante essentielle du travail, notamment pour répondre aux exigences de faible latence, de haute fiabilité, de débit élevé et de passage à l'échelle nécessaires à la coordination d'essaims de drones.

Dans un premier temps, nous avons établi les bases théoriques nécessaires à la modélisation du drone quadrotor. Le modèle dynamique et cinématique a été développé à partir des repères spatial et corps, en intégrant les équations de mouvement, les matrices de rotation et le modèle du système moteur. Ce modèle a ensuite été implémenté et simulé sous MATLAB/Simulink afin de valider la stabilité du système de contrôle. Les résultats obtenus, notamment lors des simulations de suivi de trajectoire, ont démontré la capacité du contrôleur à assurer un vol stable, précis et robuste dans un environnement simulé.

Ensuite, une plateforme complète de simulation a été mise en place en combinant PX4, AirSim, ROS2 et QGroundControl, avec une architecture répartie entre Windows natif et WSL (Ubuntu). Les tests de communication entre modules ont confirmé la fiabilité des échanges de données à travers différents protocoles (MAVLink, DDS et API Server). Les scénarios de décollage, vol stationnaire, déplacement en ligne droite et atterrissage ont validé le bon fonctionnement du système intégré, garantissant la cohérence entre le pilote automatique, l'environnement virtuel et les outils de supervision.

Sur le plan fonctionnel, l'intégration des capteurs simulés (caméra RGB, LiDAR et caméra de profondeur) a permis d'enrichir la perception environnementale du drone. La visualisation des données via RViz a confirmé la transmission correcte des informations capteurs. L'algorithme de détection YOLOv8, combiné à OpenCV, a démontré une capacité de détection humaine en temps réel avec un taux de précision d'environ 80 % et une fréquence de traitement d'environ 20 FPS. Malgré certaines limitations dues au décalage entre les données d'entraînement et l'environnement simulé, les résultats obtenus valident la faisabilité de l'approche pour des applications de recherche et sauvetage.

Par ailleurs, la reconstruction 3D de l'environnement à l'aide d'OctoMap a permis de générer une carte volumétrique de la scène simulée à partir des images de profondeur. Les expérimentations avec différentes résolutions ont mis en évidence un compromis entre précision géométrique et coût computationnel. Ces résultats confirment la validité de la chaîne complète de traitement, depuis l'acquisition des données jusqu'à l'intégration dans la carte 3D.

L'étude théorique du réseau 5G a permis d'analyser les exigences de communication dans un système multi-agents. Les mécanismes de latence ultra-faible (URLLC), de découpage réseau (network slicing) et d'architecture virtualisée offrent des perspectives importantes pour la coordination distribuée des drones. Cependant, certaines limites subsistent, notamment la dépendance à la couverture réseau, la congestion potentielle et les défis de sécurité. Cette analyse fournit une base solide pour les futures extensions du système vers des scénarios de coordination multi-drones en environnement réel.

En conclusion, ce travail a permis de développer une plateforme de simulation intégrée combinant contrôle de vol, perception visuelle, reconstruction 3D et analyse des communications 5G. Les résultats expérimentaux démontrent la cohérence de l'architecture proposée ainsi que la faisabilité des principales fonctions du système. Les perspectives futures incluent l'amélioration des algorithmes de suivi de cible, l'optimisation de la reconstruction 3D, l'intégration d'un véritable essaim de drones coopératifs, ainsi que l'implémentation expérimentale de mécanismes de coordination avancés basés sur la 5G afin d'approcher davantage les conditions réelles d'opération.

8. Répartition des tâches de l'équipe et page de contribution individuelle

8.1 Répartition des tâches au sein de l'équipe

Dans le cadre de ce projet, les tâches ont été réparties entre les membres de l'équipe en fonction de leurs compétences respectives, afin d'assurer une progression efficace et une bonne coordination globale.

- **QIAN Boda** : responsable principal du développement du système, incluant la mise en place de la plateforme de simulation (Chapitre 4 PX4 + AirSim + ROS2 + QGC), le développement des fonctionnalités principales (Chapitre 5) et la réalisation des expérimentations et analyses (Chapitre 6).
- **HUANG Bowen** : responsable de l'assistance technique et de la collecte des données, en charge de la modélisation du drone et du contrôle de vol de base, le test des modules de communication entre les composants du système, la vérification de la fiabilité du modèle Simulink, la compilation et l'organisation des données d'expérimentation, assurant la cohérence des travaux entre les différentes parties du projet.
- **NDANKOU BOBO THIERRY** : responsable de l'étude théorique de la 5G et de son adaptation à la coordination multi-robots. Il a développé un système de formation à trois robots, mis en place une communication 5G simulée avec latence et perte de paquets, conçu un algorithme d'évitement de collision basé sur le GPS et le compas, et participé à la rédaction du rapport. Il a également joué un rôle de coordination au sein de l'équipe afin d'assurer le suivi du projet.

En parallèle, certaines tâches transversales, telles que la structuration globale du rapport, la mise en forme du document et la coordination des différentes parties, ont été réalisées collectivement.

8.2 Contribution individuelle

QIAN Boda

Dans ce projet, ma contribution personnelle (**QIAN Boda**) a été majoritaire et s'est concentrée sur les aspects techniques et expérimentaux du système.

Mes principales contributions sont les suivantes :

- Développement et implémentation de la plateforme de simulation complète (intégration de PX4, AirSim, ROS2 et QGroundControl) ;
- Réalisation de la communication entre les modules et configuration des protocoles (DDS, MAVLink, TCP/UDP) ;
- Développement des fonctionnalités principales du système (Chapitre 5), incluant :
 - l'intégration des capteurs simulés (caméra, LiDAR, caméra de profondeur) ;
 - la mise en œuvre du système de détection de personnes basé sur YOLO et OpenCV ;
 - la reconstruction 3D de l'environnement à l'aide d'OctoMap ;
- Conception et réalisation des expériences (Chapitre 6), incluant :
 - les tests de stabilité du vol du drone ;
 - l'évaluation des performances de détection (précision et FPS) ;
 - l'analyse de la reconstruction 3D sous différentes résolutions ;
- Conception des supports visuels du projet (affiche et flyer de présentation) ;
- Participation à la structuration du rapport (définition du plan global, rédaction des Chapitres 1 3 4 5 6 et 7 en partie).

HUANG Bowen

Dans ce projet, j'ai assumé la responsabilité du module de contrôle, en étant en charge intégral de la modélisation du drone et de la conception du contrôle de vol. Mes travaux ont été centraux pour garantir la fiabilité, la stabilité et la précision du système de contrôle du drone, servant de fondement technique essentiel à l'ensemble du projet.

Mes principales contributions sont les suivantes :

- Modélisation théorique et mise en place du modèle du drone
 - Recherche et établissement des bases théoriques du drone quadricoptère, incluant les principes de fonctionnement des multicopters et les lois de la dynamique appliquée au vol.
 - Méné la modélisation complète et précise du drone. Intégration de différents sous-modèles : modèle du groupe moteur, modèle d'efficacité de commande, ainsi que modèles dynamique et cinématique du corps rigide.
- Théorie et conception de la technique de contrôle du drone
 - Recherche et maîtrise des théories de contrôle automatique appliquées aux drones, notamment la théorie des régulateurs et la gestion de la non-linéarité des systèmes.
 - Conception de l'architecture de commande en cascade adaptée au drone quadricoptère, en s'appuyant sur le modèle de drone établi. Et mise en place du contrôle de position en boucle extérieure et du contrôle d'attitude en boucle intérieure, tous deux basés sur des régulateurs PID série.
- Simulation et validation du contrôle de vol
 - Construction du modèle de simulation global sur Simulink, en s'appuyant sur le module de contrôle développé.
 - Réalisation de la co-simulation avec FlightGear pour assurer la visualisation 3D du vol du drone.
 - Vérification des performances du contrôle de position et d'attitude par le test de suivi de trajectoire circulaire.

Thierry NDANKOU BOBO

Pendant le projet j'ai eu la tâche de relancer mes collègues afin qu'on ne stagne pas tout ceci en demandant la progression de chacun, essayer de comprendre leurs parties et de s'adapter surtout qu'au début nous avons commencé à 4 mais terminé à 3.

Ma partie dans le projet était :

- Etude théorique sur la 5G
 - Pourquoi la 5G est adaptée à la coordination
 - Architecture complète de la 5G
 - Evaluation des performances nécessaires pour la coordination
- Développement d'un système de coordination multi-robots simulant une formation
 - Localisation, communication 5G, évitement de collision et contrôle de mouvement.
 - Mise en place d'une formation à trois robots e-puck en définissant les positions cibles relatives pour chacun (j'ai utilisé des robots roulant ici à la place des drones afin de passer le comportement physique du drone qui n'est pas pris en compte dans cette partie).
- Mise en place d'une communication 5G simulée
 - Simulation d'une communication 5G en essayant d'appliquer les conditions réalistes ; latence (20 ms), probabilité de perte de paquet
- Développement d'un algorithme d'évitement de collision grâce aux données du GPS et à l'aide du compas
- Participation à la conception des supports des rapports

RÉFÉRENCES

- [1] A. Valsan, P. B., V. D. G. H., R. S. Unnikrishnan, P. K. Reddy, and V. A., ‘Unmanned aerial vehicle for search and rescue mission’, in *2020 4th International Conference on Trends in Electronics and Informatics (ICOEI)(48184)*, Jun. 2020, pp. 684–687. doi: 10.1109/ICOEI48184.2020.9143062.
- [2] K. Namuduri, ‘Flying cell towers to the rescue’, *IEEE Spectr.*, vol. 54, no. 9, pp. 38–43, Sep. 2017, doi: 10.1109/MSPEC.2017.8012238.
- [3] C. Larsson, *5G networks: Planning, design and optimization*. Academic Press, 2018.
- [4] A. Osseiran, J. F. Monserrat, and P. Marsch, *5G mobile and wireless communications technology*. Cambridge University Press, 2016.
- [5] A. Zaidi, F. Athley, J. Medbo, U. Gustavsson, G. Durisi, and X. Chen, *5G physical layer: Principles, models and technology components*. Academic Press, 2018.
- [6] A. Albanese, V. Sciancalepore, and X. Costa-Pérez, ‘SARDO: An automated search-and-rescue drone-based solution for victims localization’, *IEEE Trans. Mob. Comput.*, vol. 21, no. 9, pp. 3312–3325, Sep. 2022, doi: 10.1109/TMC.2021.3051273.
- [7] N. Alsaedi *et al.*, ‘Smart drones in search and rescue: Leveraging AI for autonomous decision-making’, in *2025 International Conference on Activity and Behavior Computing (ABC)*, Apr. 2025, pp. 1–10. doi: 10.1109/ABC64332.2025.11118531.
- [8] B. Sanjana, R. S. Dopathi, D. Videeksha, C. S. Chowdary, R. Pitchai, and T. S. Reddy, ‘YOLO-powered deep learning framework for smart drone surveillance in emergency rescue operation’, in *2025 6th International Conference on Inventive Research in Computing Applications (ICIRCA)*, Jun. 2025, pp. 1918–1924. doi: 10.1109/ICIRCA65293.2025.11089883.